Distributional Regression

ACTL3143 & ACTL5111 Deep Learning for Actuaries

Patrick Laub

Lecture Outline

- **Introduction**
- Traditional Regression
- Stochastic Forecasts
- GLMs and Neural Networks
- Combined Actuarial Neural Network
- Mean-Variance Estimation Network
- Mixture Density Network
- Distributional Refinement Network
- Uncertainty and Regularisation
- Dropout and Ensembles

Why actuaries avoid neural networks

They're not *inherently interpretable*, so we just have to look at inputs and outputs from the black box.

Neural network models typically just output a prediction without any sense of its *confidence*.

Therefore we cannot *trust* the neural network models, which is a **dealbreaker**.

Now let's focus on actuarial problems.

Car insurance

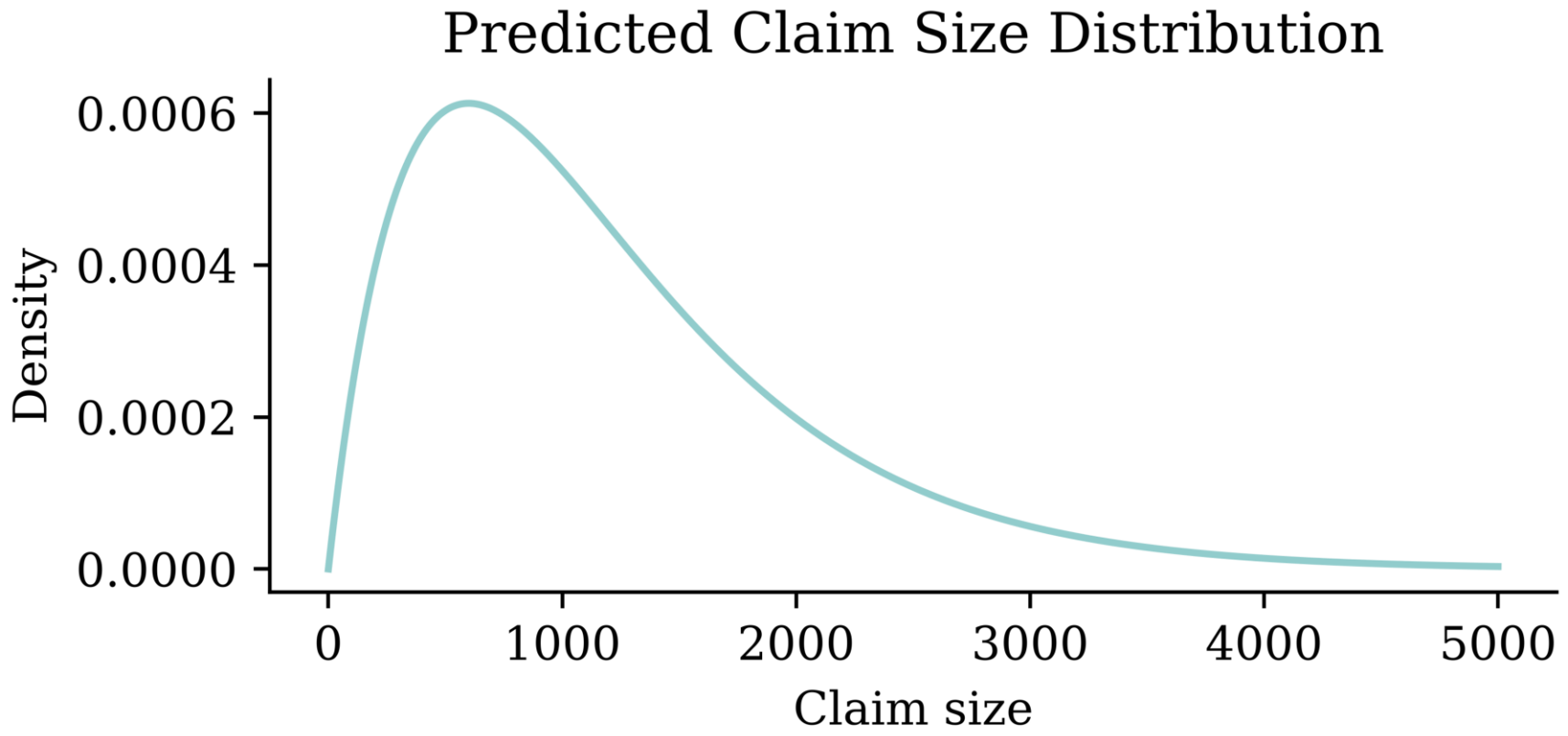
Claim size prediction

👤 Age	🚗 Age	🏎️ Type		Cost
25	3	🚙 Sedan		💵 $1,200
40	5	🚐 SUV	⟶	💵 $2,500
19	1	🏎️ Sports Car		💵 $3,800
60	10	🚗 Hatchback		💵 $800

What's wrong? Not enough rows? Not enough columns?

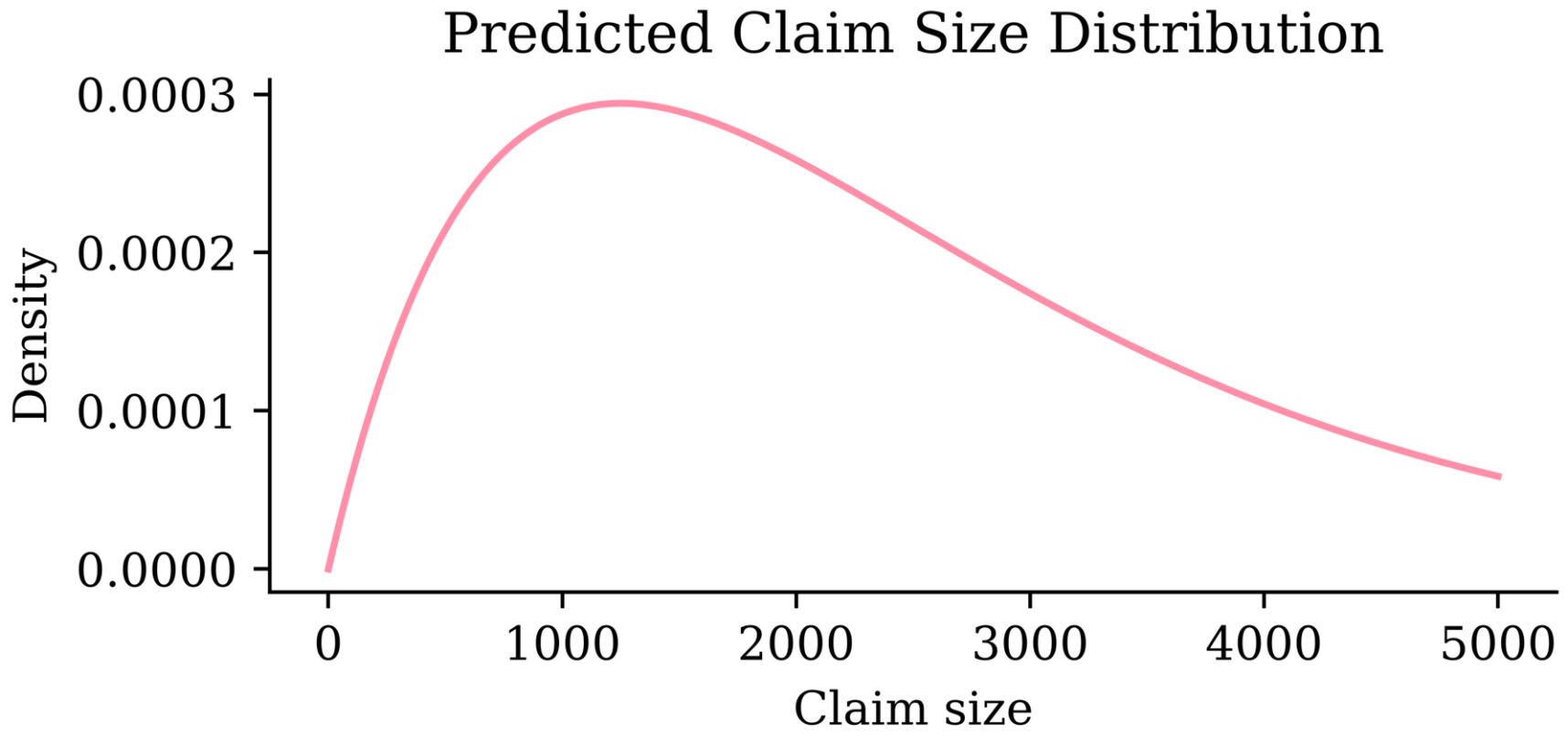
Distributional regression

Customer 1 = (25, 3, 🚙)



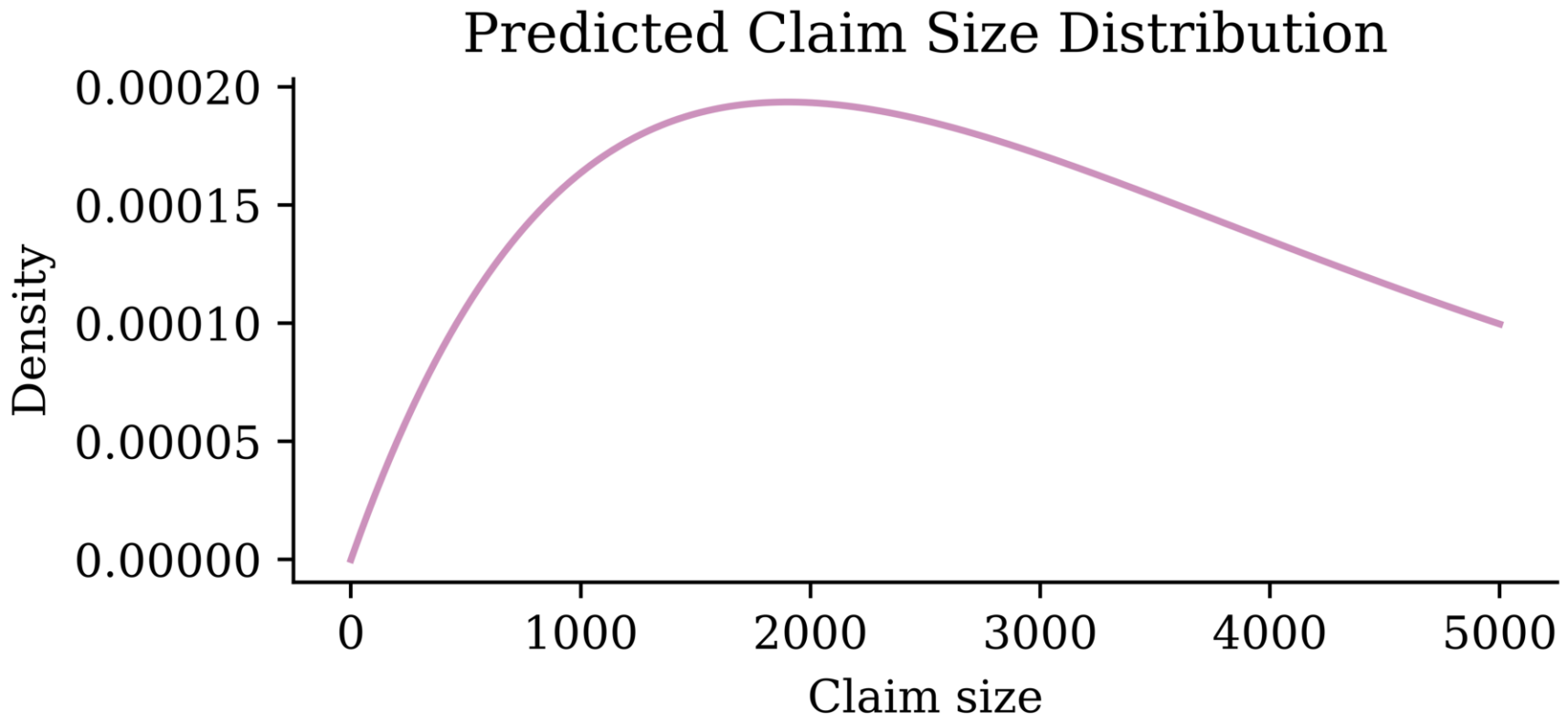
Distributional regression

Customer 2 = (40, 5, 🚐)



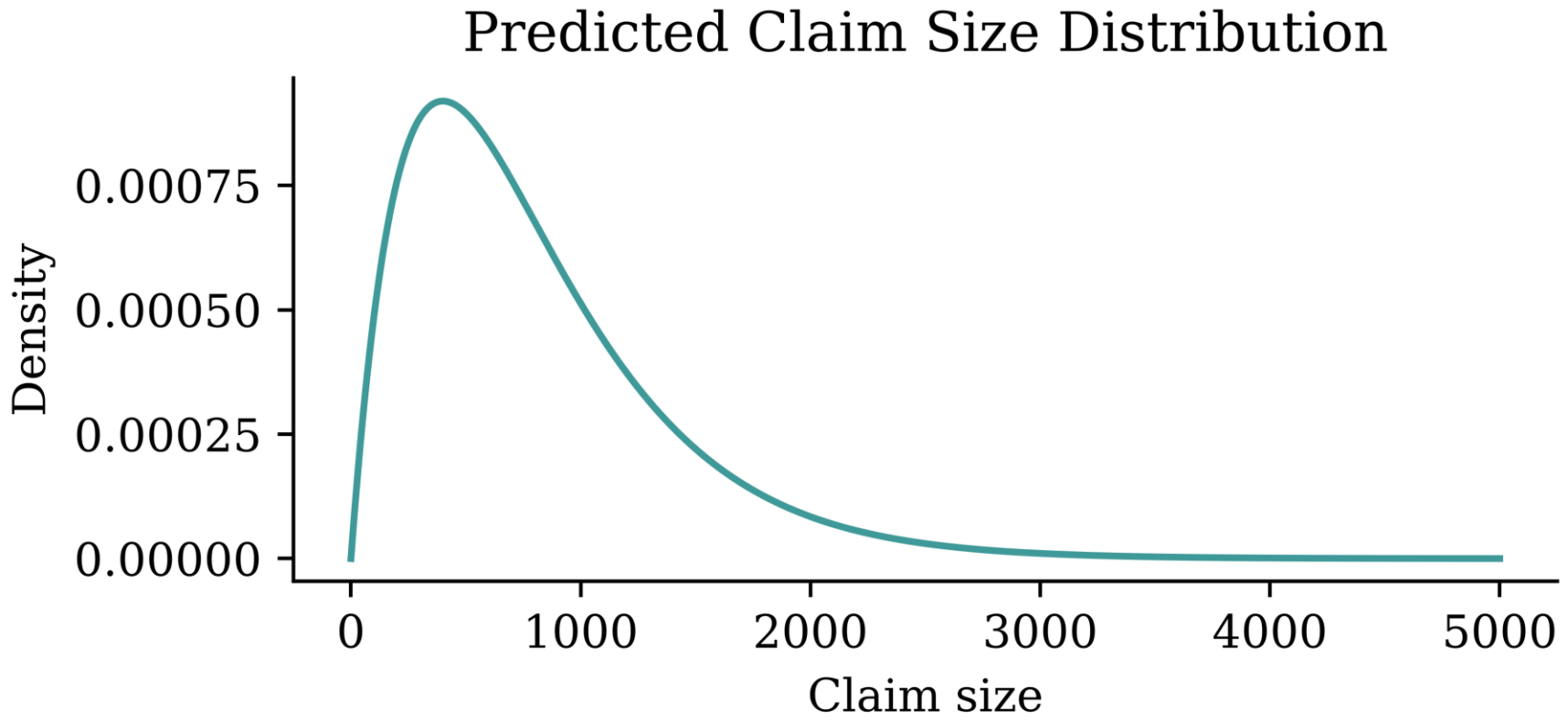
Distributional regression

Customer 3 = (19, 1, 🏎)



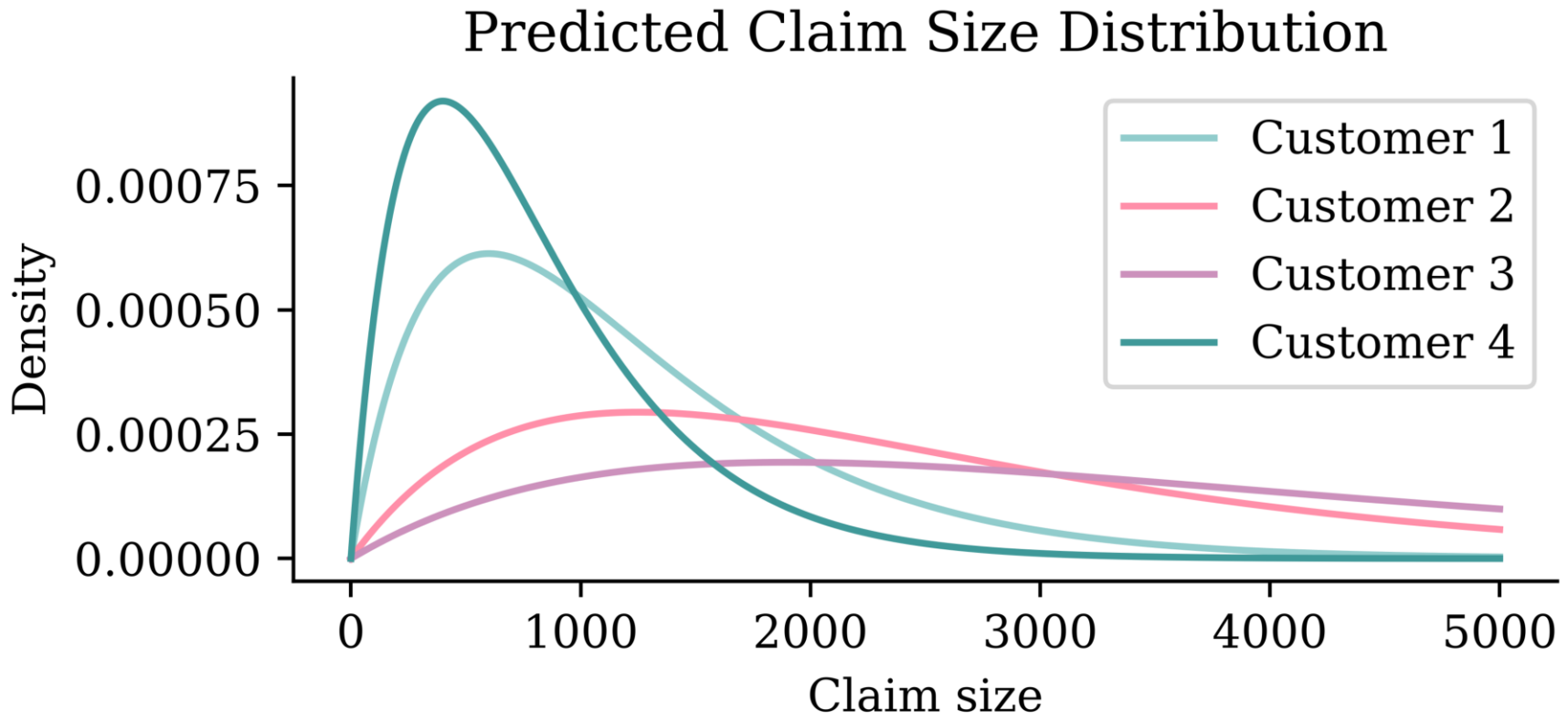
Distributional regression

Customer 4 = (60, 10, 🚗)



Distributional regression

All customers

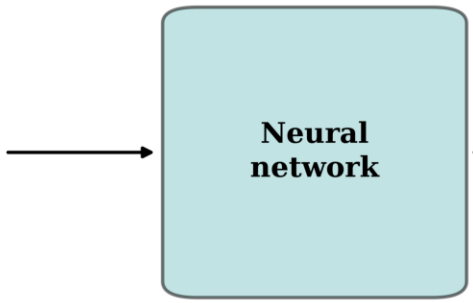


One model, many distributions

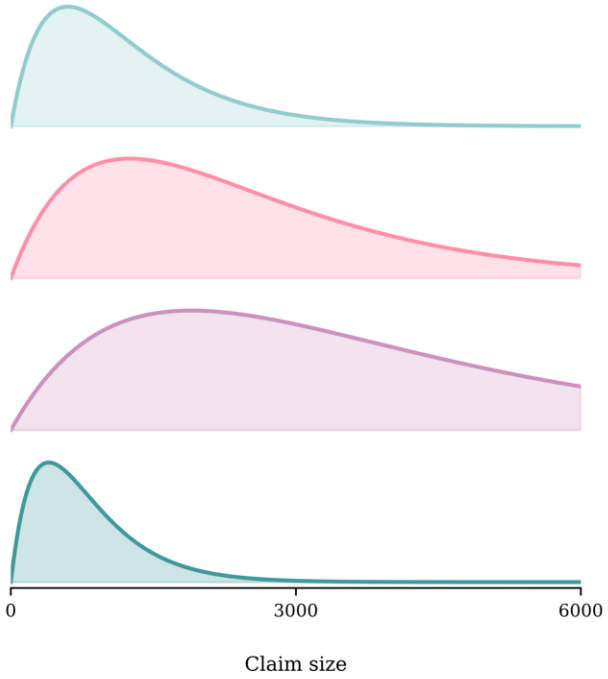
Feed in a *batch* of customers and the model returns a predicted claim-size distribution for *each* one.

Batch of data

Driver age	Car age	Type
25	3	Sedan
40	5	SUV
19	1	Sports
60	10	Hatch

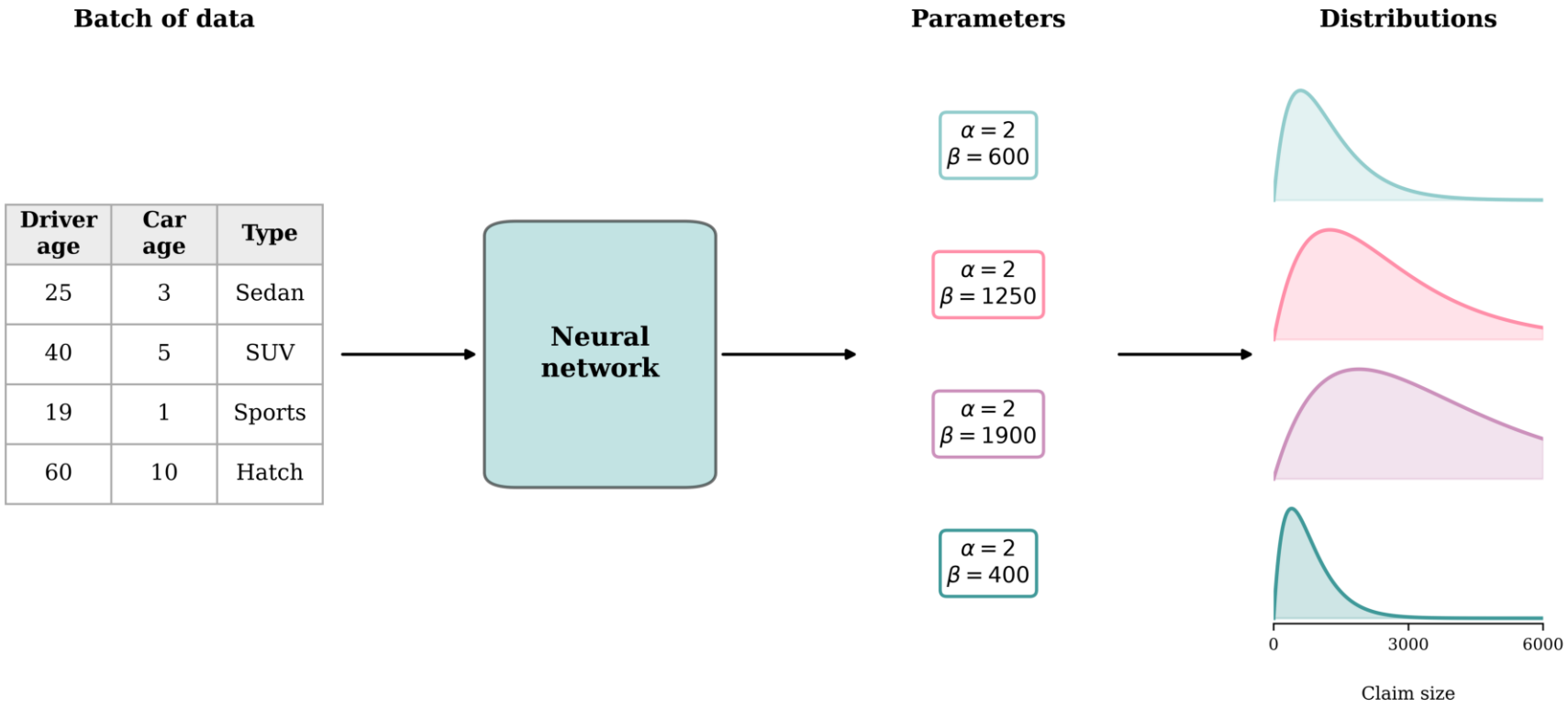


Distributions



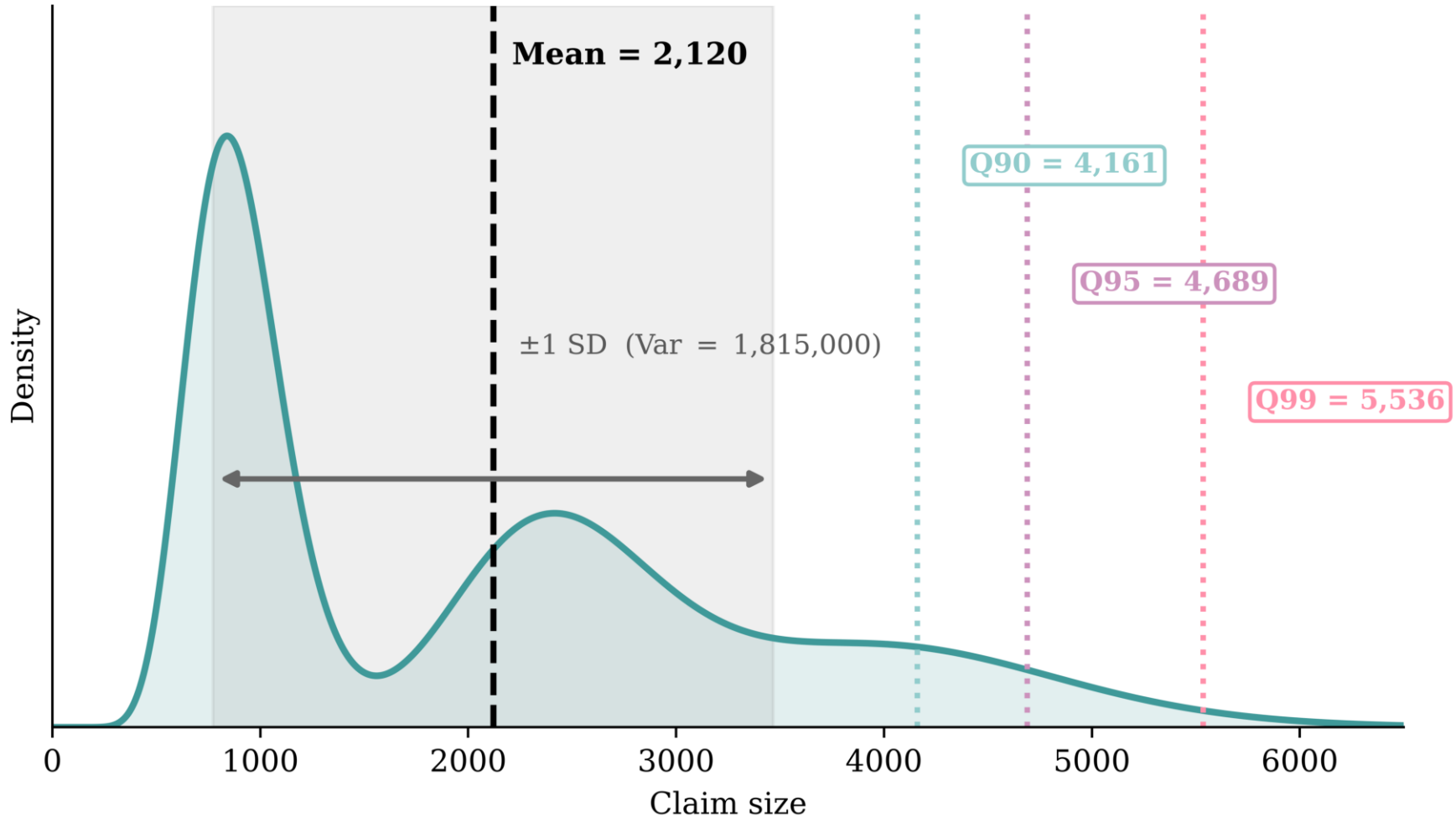
Predicting distribution parameters

In practice the network doesn't emit a whole curve — it emits the *parameters* of a chosen distribution family, which then define the distribution.



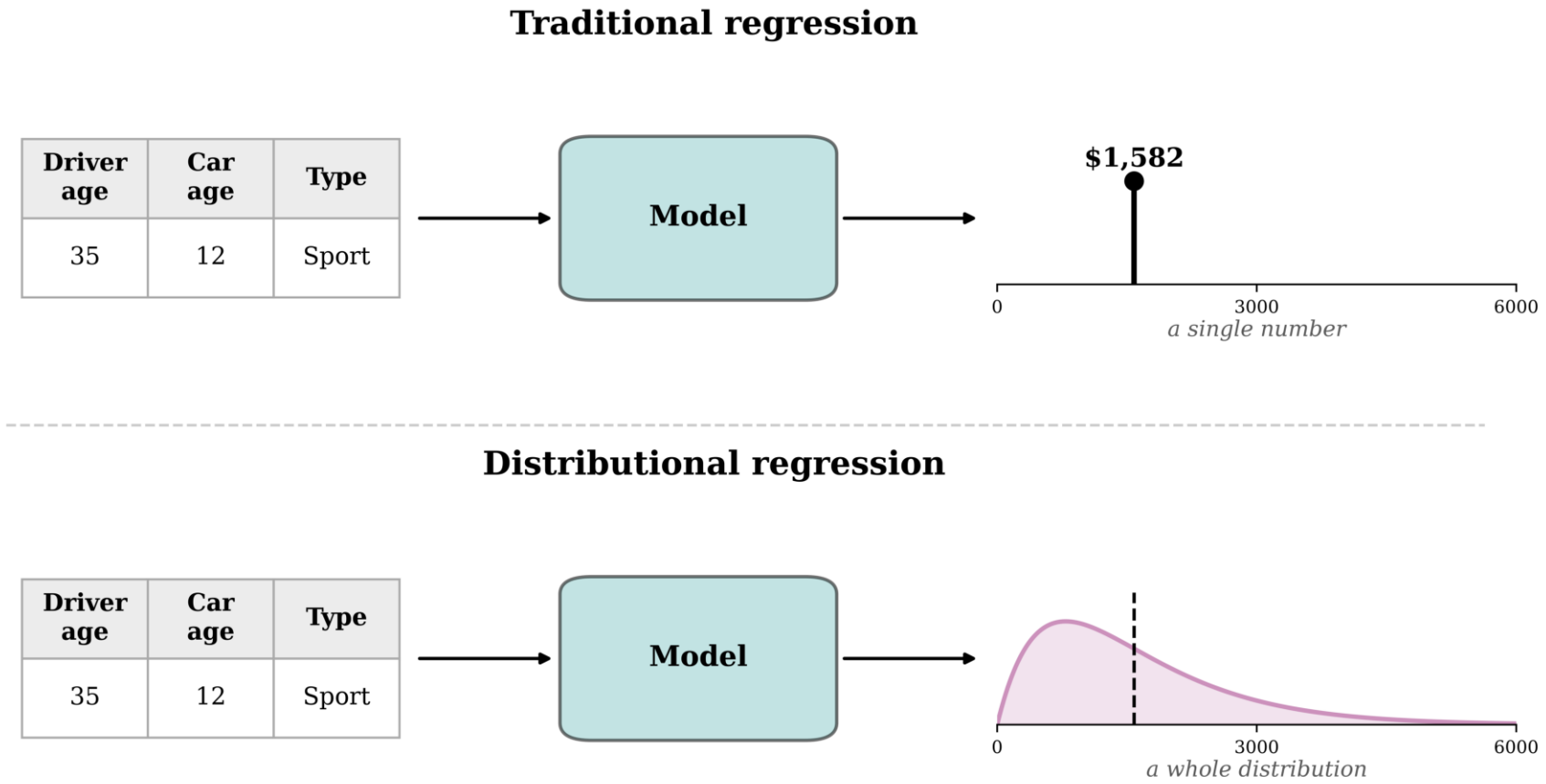
Anatomy of a predicted distribution

From a *single* predicted distribution we can read off the *mean*, the *variance*, and high *quantiles* (e.g. Value-at-Risk).



Traditional vs distributional regression

Same inputs, same model — the difference is what comes *out*.



Baseline: a generalised linear model

A gamma GLM with a log link function:

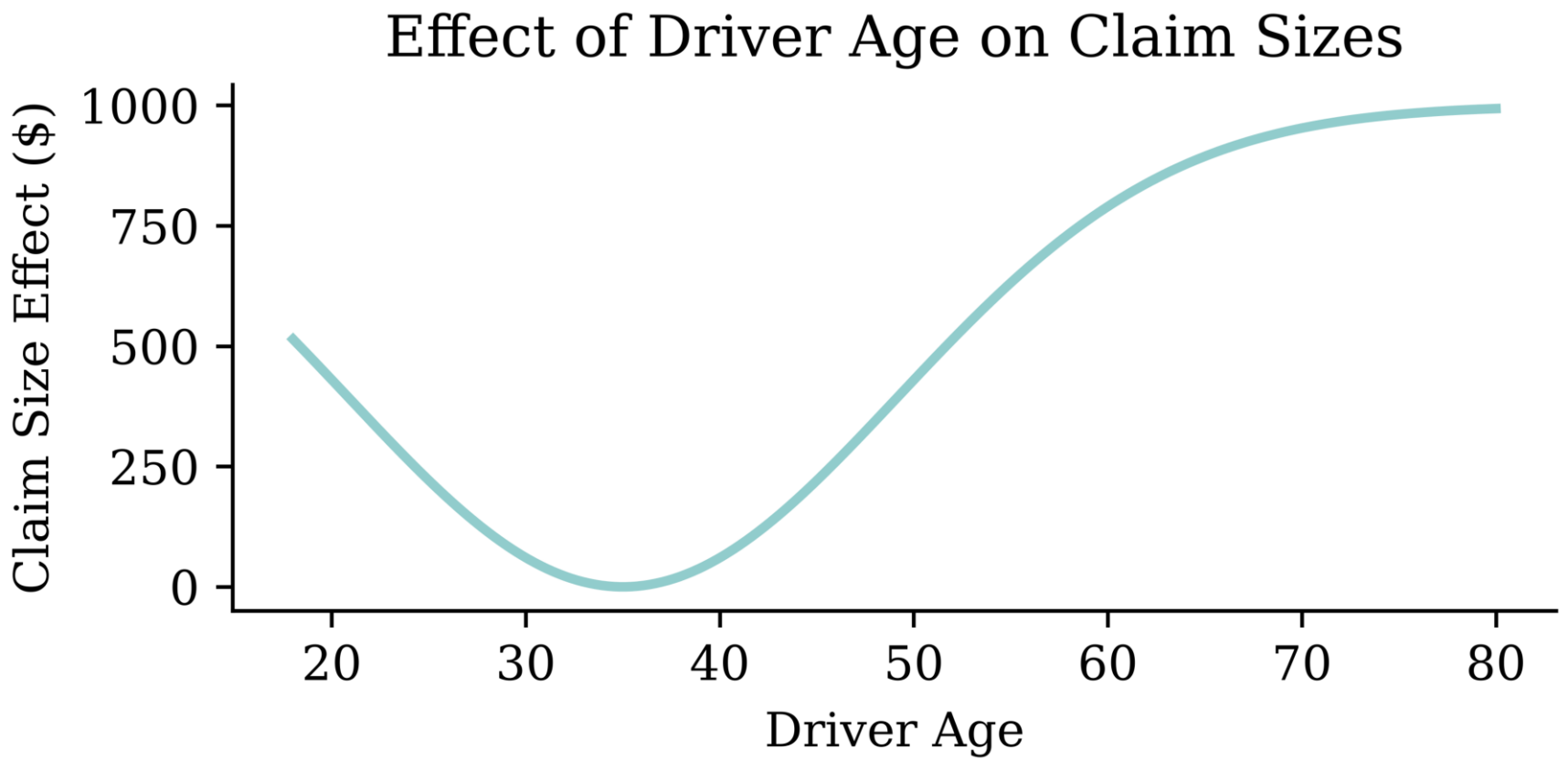
$$Y|\mathbf{X} \sim \text{Gamma}(\ldots,\ldots)$$
$$\mathbb{E}[Y|\mathbf{X}] = \exp\Big\{\beta_0 + \beta_1 \cdot \text{Age} + \beta_2 \cdot \text{Car Age} + \beta_3 \cdot \text{Type}\Big\}$$

A simple model, easy to train and interpret, but...

GLMs can be

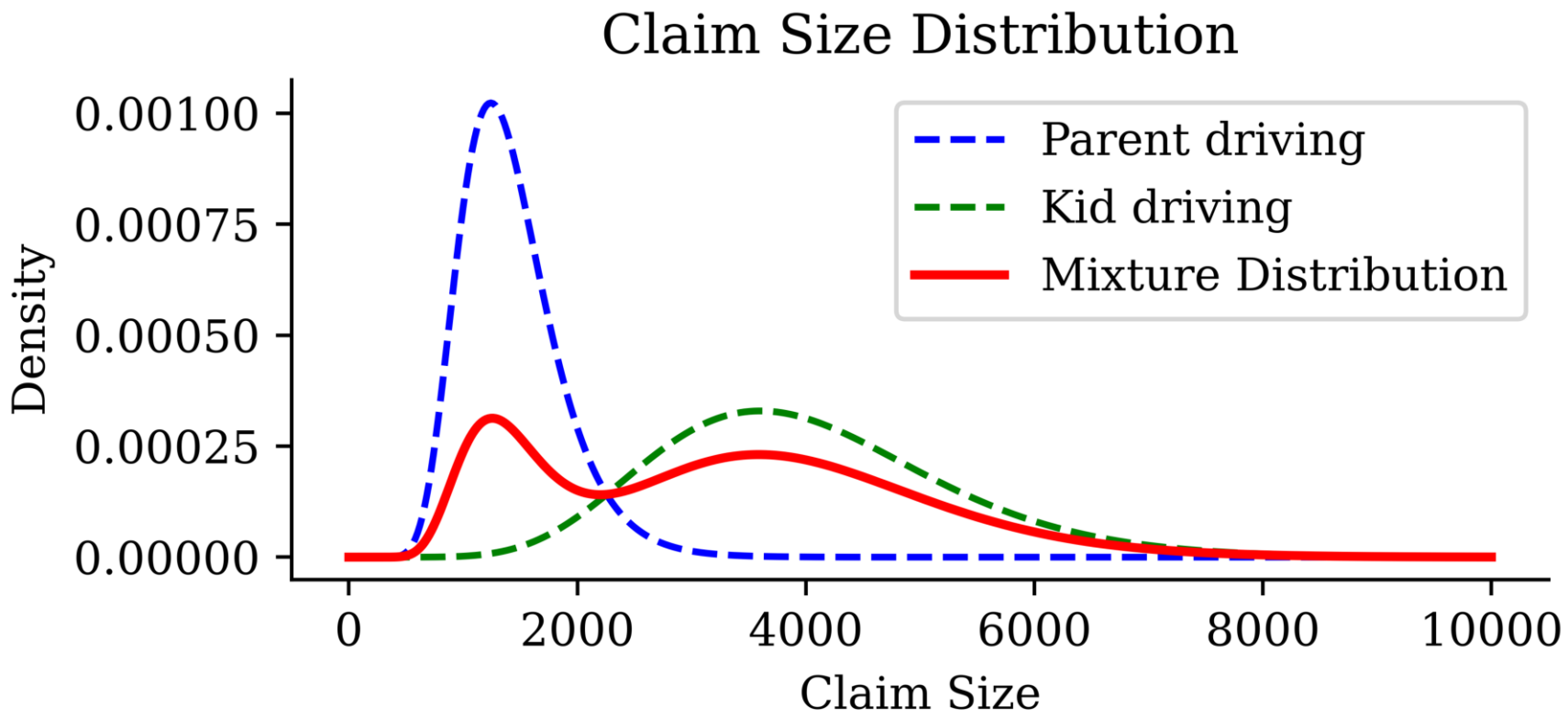
1. Bad at *regression*
2. Bad at *distributional* regression

Example 1: Non-monotonicity

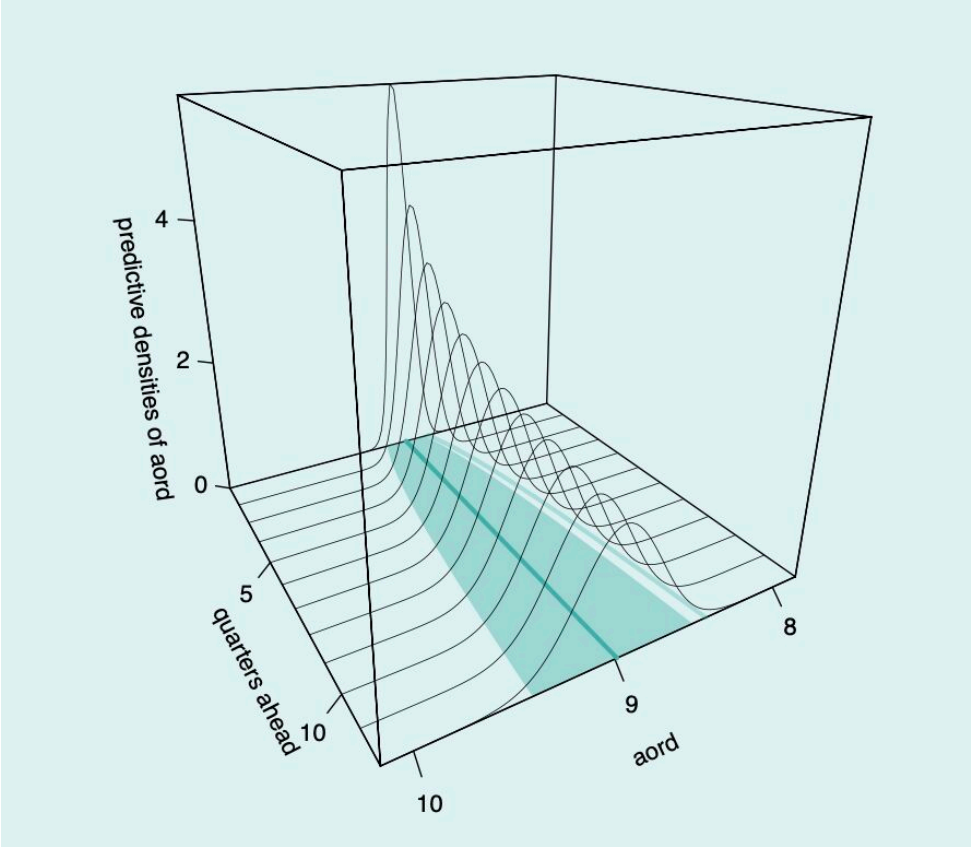


GLMs cannot (easily) do this $\longrightarrow$ Use a neural network

Example 2: Multimodality



Key idea



An example of distributional forecasting over the All Ordinaries Index

- Earlier machine learning models focused on point estimates.
- However, in many applications, we need to understand the distribution of the response variable.
- Each prediction becomes a *distribution* over the possible outcomes

Source: Tomasz Woźniak (2024), LinkedIn Post, accessed on July 15 2024.

We will focus on regression not classification

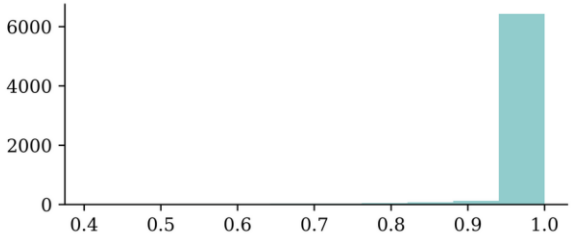
Classifiers already give us a probability, which is a big step up compared to regression models.

However, neural networks' "probabilities" can be overconfident.

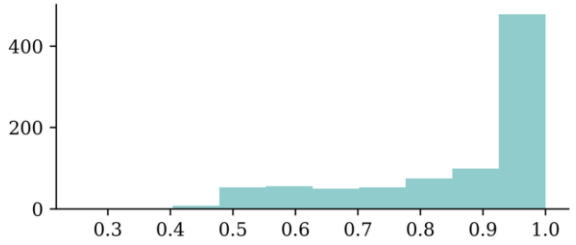
Confidence of predictions

```
y_pred = keras.ops.convert_to_numpy(keras.activations.softmax(model(X_test)))
y_pred_class = np.argmax(y_pred, axis=1)
y_pred_prob = y_pred[np.arange(y_pred.shape[0]), y_pred_class]

confidence_when_correct = y_pred_prob[y_pred_class == y_test]
confidence_when_wrong = y_pred_prob[y_pred_class ≠ y_test]
```

```
plt.hist(confidence_when_correct);
```




```
plt.hist(confidence_when_wrong);
```



We already saw a case of this.

See Guo et al. (2017).

Lecture Outline

- Introduction
- **Traditional Regression**
- Stochastic Forecasts
- GLMs and Neural Networks
- Combined Actuarial Neural Network
- Mean-Variance Estimation Network
- Mixture Density Network
- Distributional Refinement Network
- Uncertainty and Regularisation
- Dropout and Ensembles

Package imports

```
import random
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

import keras
from keras.models import Sequential, Model
from keras.layers import Input, Dense, Concatenate, Dropout
from keras.callbacks import EarlyStopping
from keras.initializers import Constant
from keras.regularizers import L1, L2

from sklearn.model_selection import train_test_split
from sklearn.compose import make_column_transformer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler, OrdinalEncoder
from sklearn.linear_model import LinearRegression
from sklearn.datasets import fetch_california_housing
from sklearn import set_config
set_config(transform_output="pandas")

import scipy.stats as stats
import statsmodels.api as sm
from torch.distributions import Gamma, MixtureSameFamily, Categorical
```

Notation

- scalars are denoted by lowercase letters, e.g., y,
- vectors are denoted by bold lowercase letters, e.g.,

$$\boldsymbol{y} = (y_1, \ldots, y_n),$$

- random variables are denoted by capital letters, e.g., Y
- random vectors are denoted by bold capital letters, e.g.,

$$\boldsymbol{X} = (X_1, \ldots, X_p),$$

- matrices are denoted by bold uppercase non-italics letters, e.g.,

$$\mathbf{X} = \begin{pmatrix} x_{11} & \cdots & x_{1p} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{np} \end{pmatrix}.$$

Regression notation

- n is the number of observations, p is the number of features,
- the true coefficients are $\boldsymbol{\beta} = (\beta_0, \beta_1, \ldots, \beta_p)$,
- β_0 is the intercept, $\beta_1, \ldots, \beta_p$ are the coefficients,
- $\boldsymbol{\beta}^*$ is the estimated coefficient vector,
- $\boldsymbol{x}_i = (1, x_{i1}, x_{i2}, \ldots, x_{ip})$ is the feature vector for the ith observation,
- y_i is the response variable for the ith observation,
- $\hat{y}_i$ is the predicted value for the ith observation,

Distributional assumptions become loss functions

Each example below follows the same maximum-likelihood recipe:

1. Choose a distribution and write its density or mass function.

$$Y_i | \boldsymbol{X} = \boldsymbol{x}_i \sim F(\theta_i), \qquad f(y_i; \theta_i)$$

2. Multiply the probabilities to form the likelihood.

$$L(\boldsymbol{\beta}) = \prod_{i=1}^{n} f(y_i; \theta_i)$$

3. Take logs to get the log-likelihood and negate it to get negative log likelihood.

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^{n} \log f(y_i; \theta_i) \quad \Rightarrow \text{NLL}(\boldsymbol{\beta}) = -\ell(\boldsymbol{\beta})$$

4. Drop constants and rescale to get the model loss.

$$\text{loss}(\boldsymbol{y}, \hat{\boldsymbol{y}})$$

Traditional regression

Multiple linear regression assumes the data-generating process is

$$Y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_p x_{ip} + \varepsilon$$

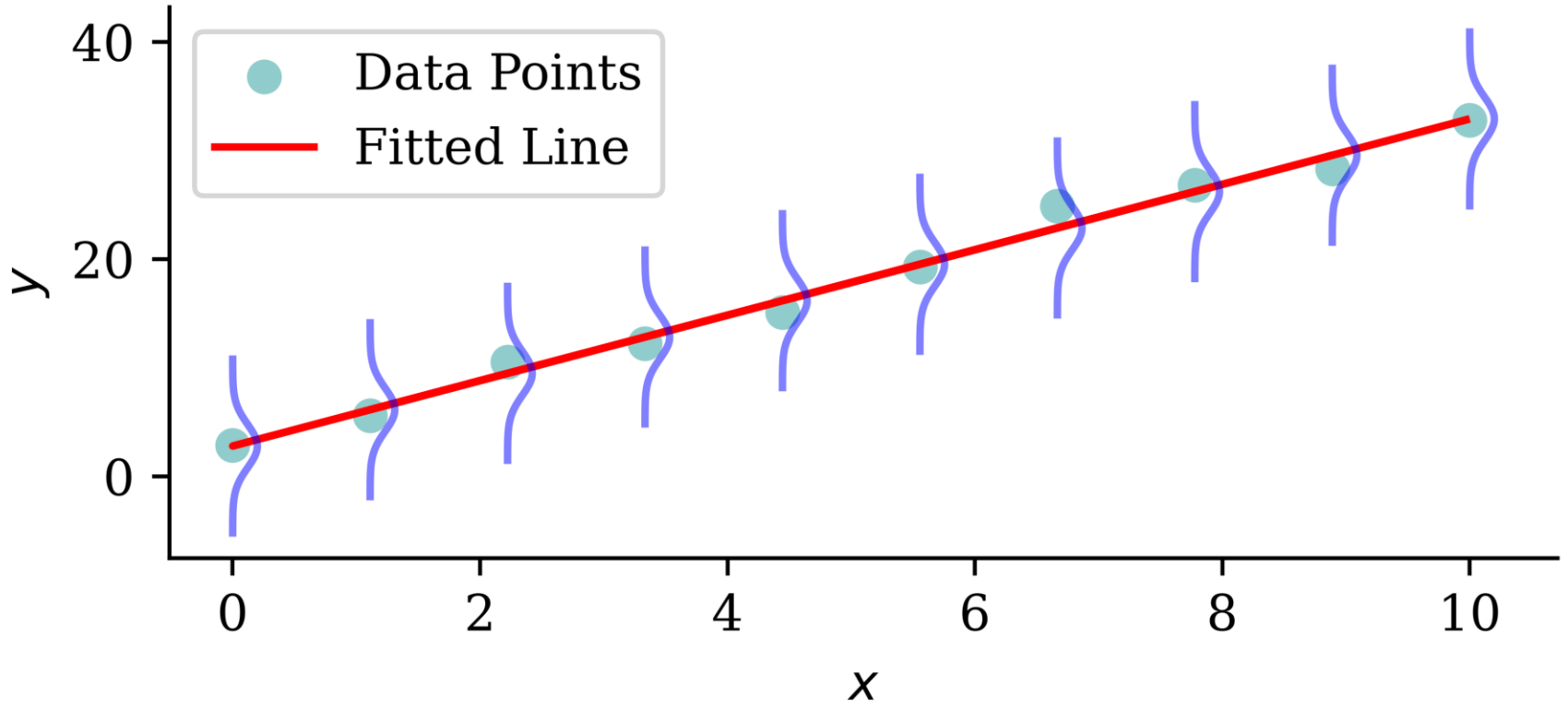
where $\varepsilon \sim \mathcal{N}(0, \sigma^2)$.

We estimate the coefficients $\beta_0, \beta_1, \ldots, \beta_p$ by minimising the sum of squared residuals or mean squared error

$$\text{RSS} := \sum_{i=1}^{n} (y_i - \hat{y}_i)^2, \quad \text{MSE} := \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2,$$

where $\hat{y}_i$ is the predicted value for the ith observation.

Visualising the distribution of each Y



The probabilistic view

$$Y_i \sim \mathcal{N}(\mu_i, \sigma^2)$$

where $\mu_i = \beta_0 + \beta_1 x_{i1} + \ldots + \beta_p x_{ip}$, and the σ^2 is known.

The $\mathcal{N}(\mu, \sigma^2)$ normal distribution has p.d.f.

$$f(y) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y-\mu)^2}{2\sigma^2}\right).$$

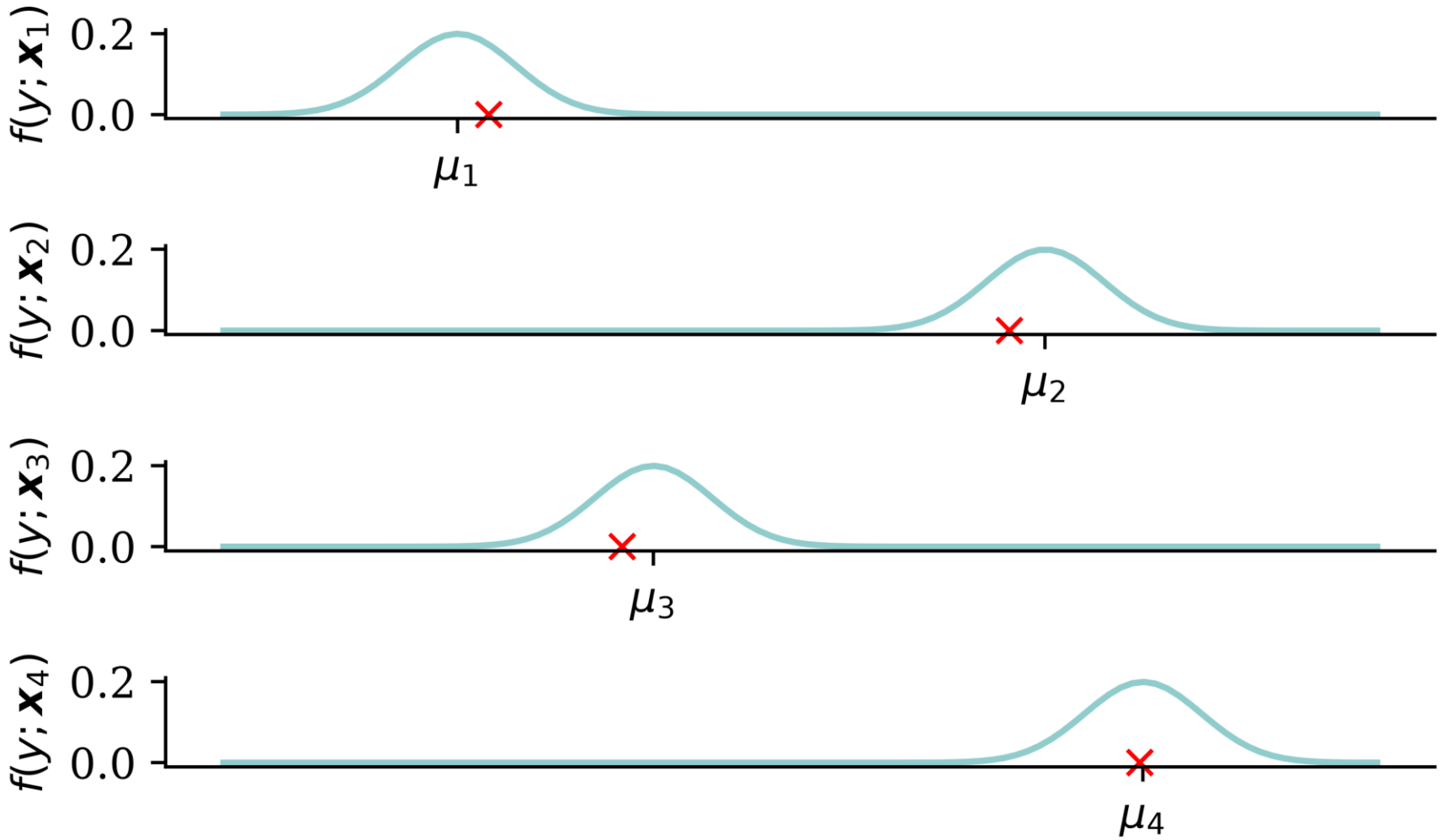
The likelihood function is

$$L(\boldsymbol{\beta}) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i-\mu_i)^2}{2\sigma^2}\right)$$

$$\Rightarrow \ell(\boldsymbol{\beta}) = -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log(\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i-\mu_i)^2.$$

Perform maximum likelihood estimation to find $\boldsymbol{\beta}$.

The predicted distributions



The machine learning view

The negative log-likelihood $\text{NLL}(\boldsymbol{\beta}) := -\ell(\boldsymbol{\beta})$ is to be minimised:

$$\text{NLL}(\boldsymbol{\beta}) = \frac{n}{2}\log(2\pi) + \frac{n}{2}\log(\sigma^2) + \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \mu_i)^2.$$

As σ^2 is fixed, minimising NLL is equivalent to minimising MSE:

$$\begin{aligned}
\boldsymbol{\beta}^* &= \arg\min_{\boldsymbol{\beta}} \text{NLL}(\boldsymbol{\beta}) \\
&= \arg\min_{\boldsymbol{\beta}} \frac{n}{2}\log(2\pi) + \frac{n}{2}\log(\sigma^2) + \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \mu_i)^2 \\
&= \arg\min_{\boldsymbol{\beta}} \frac{1}{n}\sum_{i=1}^{n}\Big(y_i - \hat{y}_i(\boldsymbol{x}_i; \boldsymbol{\beta})\Big)^2 \\
&= \arg\min_{\boldsymbol{\beta}} \text{MSE}\big(\boldsymbol{y}, \hat{\boldsymbol{y}}(\mathbf{X}; \boldsymbol{\beta})\big).
\end{aligned}$$

Generalised Linear Model (GLM)

The GLM is often characterised by the mean prediction:

$$\mu(\boldsymbol{x}; \boldsymbol{\beta}) = g^{-1}\left(\langle \boldsymbol{\beta}, \boldsymbol{x} \rangle\right)$$

where g is the link function.

Common GLM distributions for the response variable include:

- Normal distribution with identity link (just MLR)
- Bernoulli distribution with logit link (logistic regression)
- Poisson distribution with log link (Poisson regression)
- Gamma distribution with log link

Logistic regression

A Bernoulli distribution with parameter p has p.m.f.

$$f(y) = \begin{cases} p & \text{if } y = 1 \\ 1-p & \text{if } y = 0 \end{cases} = p^y(1-p)^{1-y}.$$

Our model is $Y|\boldsymbol{X} = \boldsymbol{x}$ follows a Bernoulli distribution with parameter

$$\mu(\boldsymbol{x}; \boldsymbol{\beta}) = \frac{1}{1 + \exp\left(-\langle \boldsymbol{\beta}, \boldsymbol{x} \rangle\right)} = \mathbb{P}(Y = 1 | \boldsymbol{X} = \boldsymbol{x}).$$

The likelihood function, using $\mu_i := \mu(\boldsymbol{x}_i; \boldsymbol{\beta})$, is

$$L(\boldsymbol{\beta}) = \prod_{i=1}^{n} \begin{cases} \mu_i & \text{if } y_i = 1 \\ 1-\mu_i & \text{if } y_i = 0 \end{cases} = \prod_{i=1}^{n} \mu_i^{y_i}(1-\mu_i)^{1-y_i}.$$

Binary cross-entropy loss

$$L(\boldsymbol{\beta}) = \prod_{i=1}^{n} \mu_i^{y_i}(1-\mu_i)^{1-y_i} \Rightarrow \ell(\boldsymbol{\beta}) = \sum_{i=1}^{n}\Big(y_i \log(\mu_i) + (1-y_i)\log(1-\mu_i)\Big).$$

The negative log-likelihood is

$$\text{NLL}(\boldsymbol{\beta}) = -\sum_{i=1}^{n}\Big(y_i \log(\mu_i) + (1-y_i)\log(1-\mu_i)\Big).$$

The binary cross-entropy loss is basically identical:

$$\text{BCE}(\boldsymbol{y}, \boldsymbol{\mu}) = -\frac{1}{n}\sum_{i=1}^{n}\Big(y_i \log(\mu_i) + (1-y_i)\log(1-\mu_i)\Big).$$

Poisson regression

A Poisson distribution with rate λ has p.m.f.

$$f(y) = \frac{\lambda^y \exp(-\lambda)}{y!}.$$

Our model is $Y|\boldsymbol{X} = \boldsymbol{x}$ is Poisson distributed with parameter

$$\mu(\boldsymbol{x}; \boldsymbol{\beta}) = \exp\left(\langle \boldsymbol{\beta}, \boldsymbol{x} \rangle\right).$$

The likelihood function is

$$L(\boldsymbol{\beta}) = \prod_{i=1}^{n} \frac{\mu_i^{y_i} \exp(-\mu_i)}{y_i!}$$

$$\Rightarrow \ell(\boldsymbol{\beta}) = \sum_{i=1}^{n} \Big(-\mu_i + y_i \log(\mu_i) - \log(y_i!) \Big).$$

Poisson loss

The negative log-likelihood is

$$\text{NLL}(\boldsymbol{\beta}) = \sum_{i=1}^{n} \Big(\mu_i - y_i \log(\mu_i) + \log(y_i!) \Big).$$

The Poisson loss is

$$\text{Poisson}(\boldsymbol{y}, \boldsymbol{\mu}) = \frac{1}{n} \sum_{i=1}^{n} \Big(\mu_i - y_i \log(\mu_i) \Big).$$

Gamma regression

A Gamma distribution with mean μ and dispersion ϕ has p.d.f.

$$f(y;\mu,\phi) = \frac{(\mu\phi)^{-\frac{1}{\phi}}}{\Gamma\left(\frac{1}{\phi}\right)} y^{\frac{1}{\phi}-1} \mathrm{e}^{-\frac{y}{\mu\phi}}$$

Our model is $Y|\boldsymbol{X} = \boldsymbol{x}$ is Gamma distributed with a dispersion of ϕ and a mean of $\mu(\boldsymbol{x};\boldsymbol{\beta}) = \exp\left(\langle \boldsymbol{\beta}, \boldsymbol{x} \rangle\right)$.

The likelihood function is

$$L(\boldsymbol{\beta}) = \prod_{i=1}^{n} \frac{(\mu_i\phi)^{-\frac{1}{\phi}}}{\Gamma\left(\frac{1}{\phi}\right)} y_i^{\frac{1}{\phi}-1} \exp\left(-\frac{y_i}{\mu_i\phi}\right)$$

$$\Rightarrow \ell(\boldsymbol{\beta}) = \sum_{i=1}^{n} \left[-\frac{1}{\phi}\log(\mu_i\phi) - \log\Gamma\left(\frac{1}{\phi}\right) + \left(\frac{1}{\phi}-1\right)\log(y_i) - \frac{y_i}{\mu_i\phi} \right].$$

Gamma loss

The negative log-likelihood is

$$\text{NLL}(\boldsymbol{\beta}) = \sum_{i=1}^{n} \left[\frac{1}{\phi} \log(\mu_i \phi) + \log \Gamma\left(\frac{1}{\phi}\right) - \left(\frac{1}{\phi} - 1\right) \log(y_i) + \frac{y_i}{\mu_i \phi} \right].$$

Since ϕ is a nuisance parameter

$$\text{NLL}(\boldsymbol{\beta}) = \sum_{i=1}^{n} \left[\frac{1}{\phi} \log(\mu_i) + \frac{y_i}{\mu_i \phi} \right] + \text{const} \propto \sum_{i=1}^{n} \left[\log(\mu_i) + \frac{y_i}{\mu_i} \right].$$

Note

As $\log(\mu_i) = \log(y_i) - \log(y_i/\mu_i)$, we could write an alternative version

$$\text{NLL}(\boldsymbol{\beta}) \propto \sum_{i=1}^{n} \left[\log(y_i) - \log\left(\frac{y_i}{\mu_i}\right) + \frac{y_i}{\mu_i} \right] \propto \sum_{i=1}^{n} \left[\frac{y_i}{\mu_i} - \log\left(\frac{y_i}{\mu_i}\right) \right].$$

Why do actuaries use GLMs?

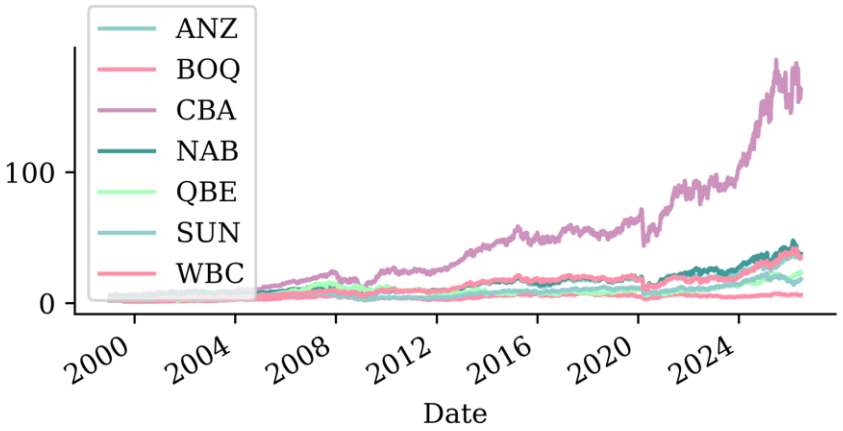
- GLMs are interpretable.
- GLMs are flexible (can handle different types of response variables).
- We get the full distribution of the response variable, not just the mean.

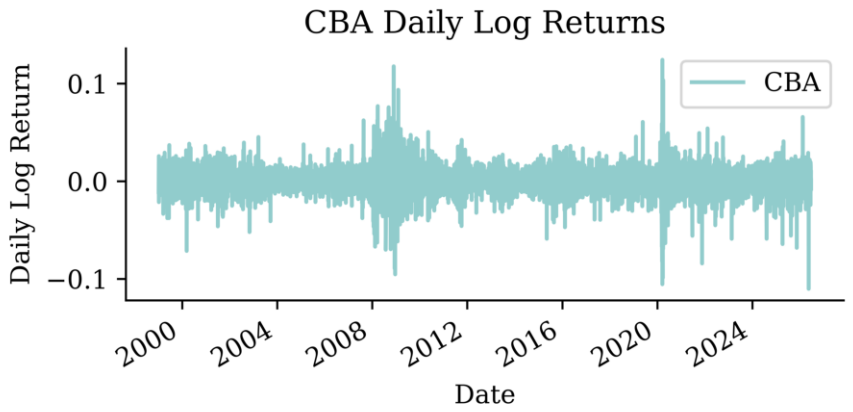
This last point is particularly important for analysing worst-case scenarios.

Lecture Outline

- Introduction
- Traditional Regression
- **Stochastic Forecasts**
- GLMs and Neural Networks
- Combined Actuarial Neural Network
- Mean-Variance Estimation Network
- Mixture Density Network
- Distributional Refinement Network
- Uncertainty and Regularisation
- Dropout and Ensembles

Stock price forecasting



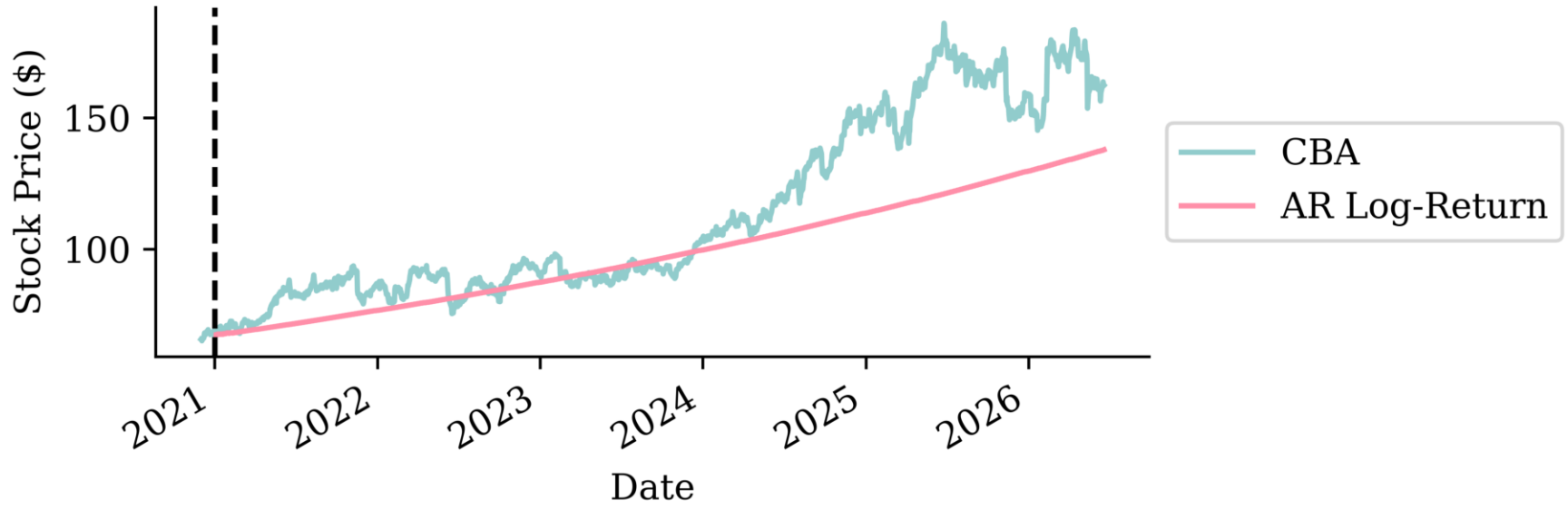


Noisy auto-regressive forecast

```python
def noisy_autoregressive_forecast(model, X_val, sigma):
    """Roll a one-step model forward, feeding each (noisy) prediction back in."""
    window = np.asarray(X_val.iloc[0], dtype="float32").copy()
    preds = []
    for _ in range(len(X_val)):
        X_next = pd.DataFrame(window.reshape(1, -1), columns=X_val.columns)
        next_value = model.predict(X_next).flatten()[0]
        next_value += np.random.normal(0, sigma)
        preds.append(next_value)
        window = np.append(window[1:], next_value)  # drop oldest, add prediction
    return pd.Series(preds, index=X_val.index, name="Multi Step")
```

Original forecast

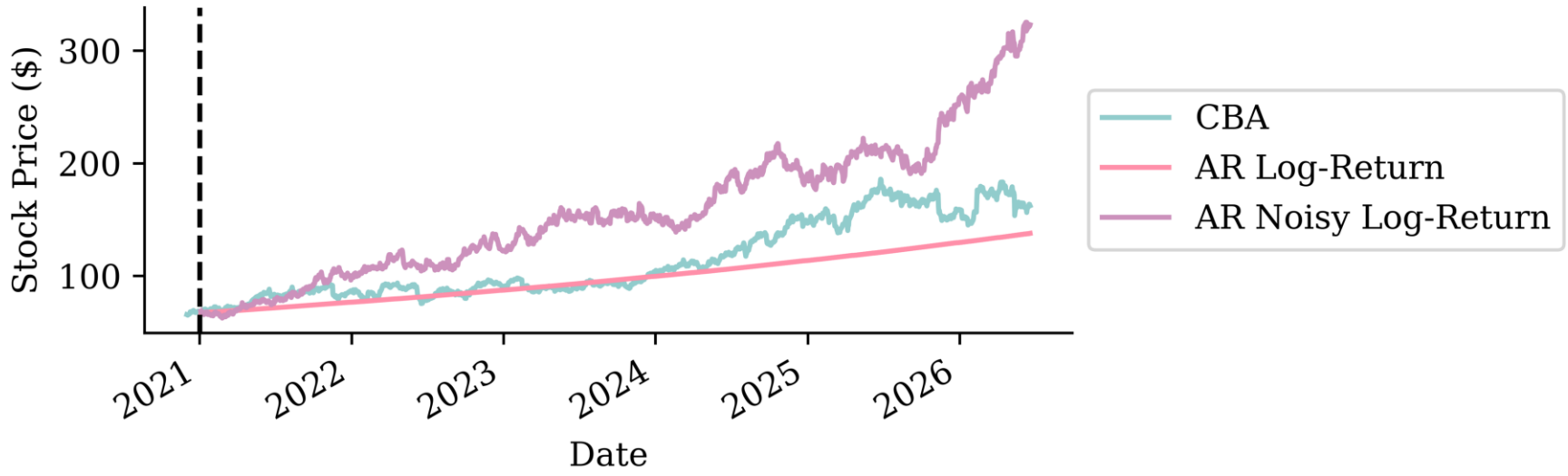
```
lr_forecast = noisy_autoregressive_forecast(lr, X_test, 0)
last_price = get_last_price(stock, cutoff_date="2020-12")
price_forecast = log_to_price(lr_forecast, last_price)
```




```
residuals = y_val - lr.predict(X_val)
sigma = np.std(residuals)
```

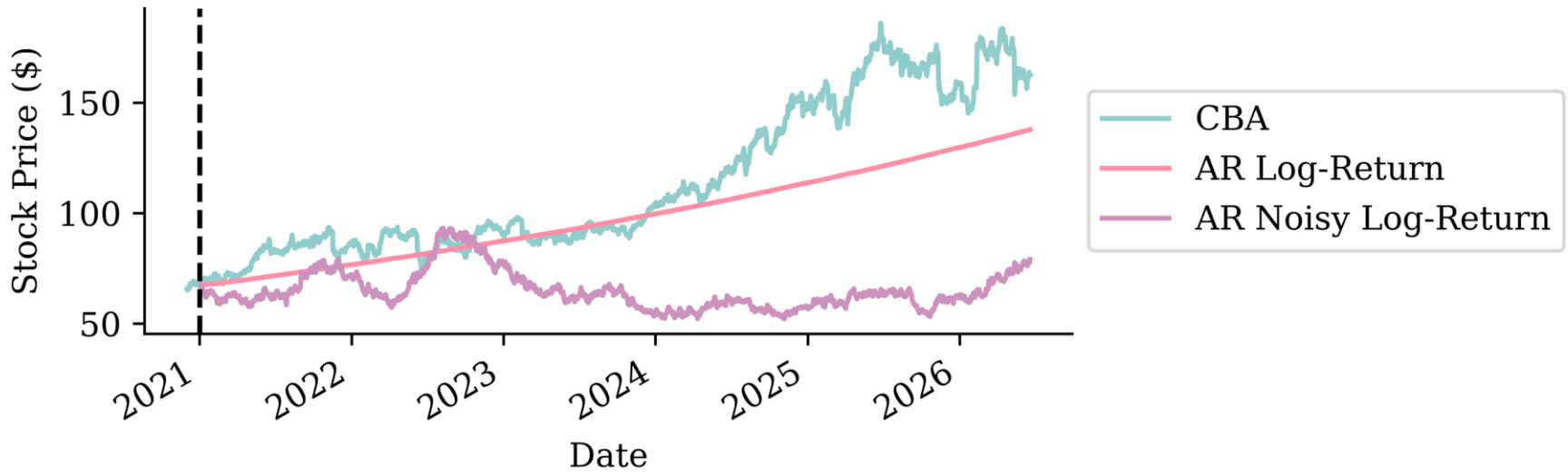
With noise

```
np.random.seed(1)
lr_noisy_forecast = noisy_autoregressive_forecast(lr, X_test, sigma)
price_noisy = log_to_price(lr_noisy_forecast, last_price)
```



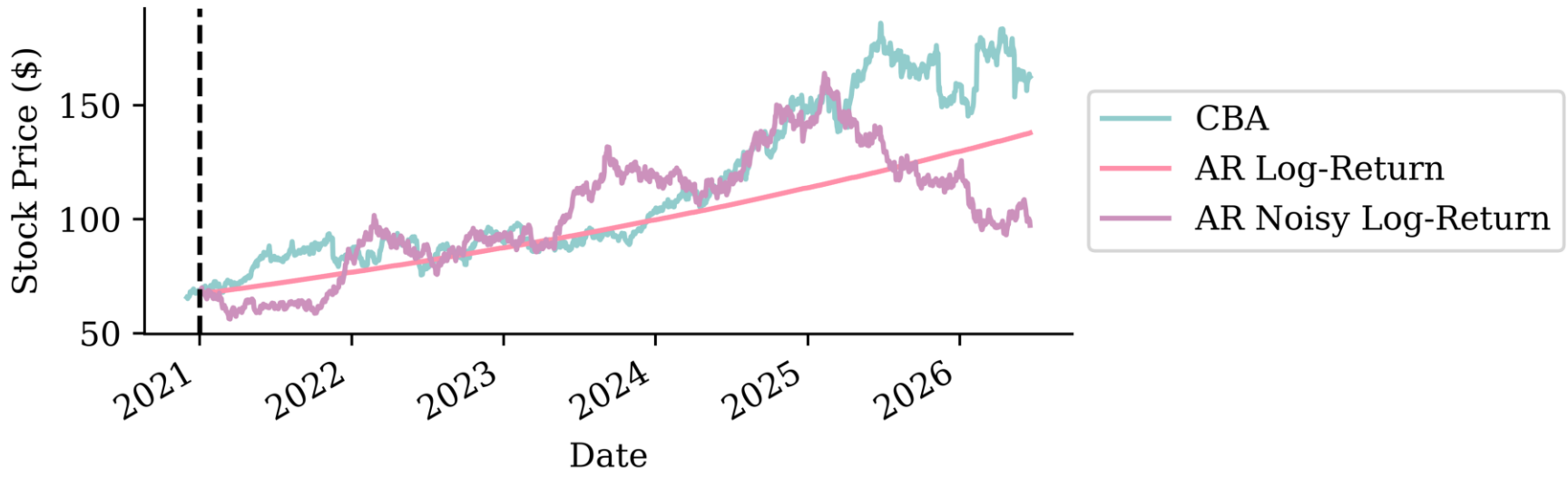
With noise

```
np.random.seed(2)
lr_noisy_forecast = noisy_autoregressive_forecast(lr, X_test, sigma)
price_noisy = log_to_price(lr_noisy_forecast, last_price)
```



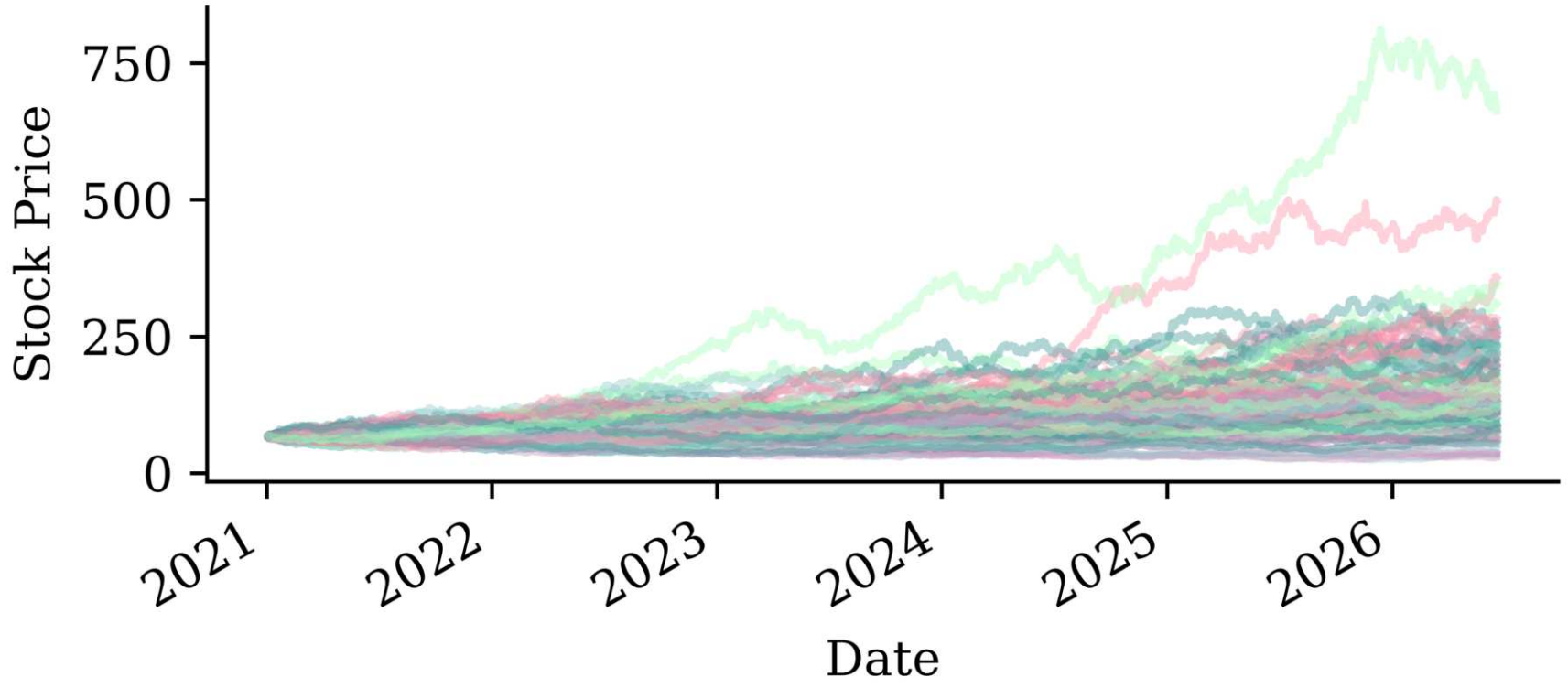
With noise

```
np.random.seed(3)
lr_noisy_forecast = noisy_autoregressive_forecast(lr, X_test, sigma)
price_noisy = log_to_price(lr_noisy_forecast, last_price)
```



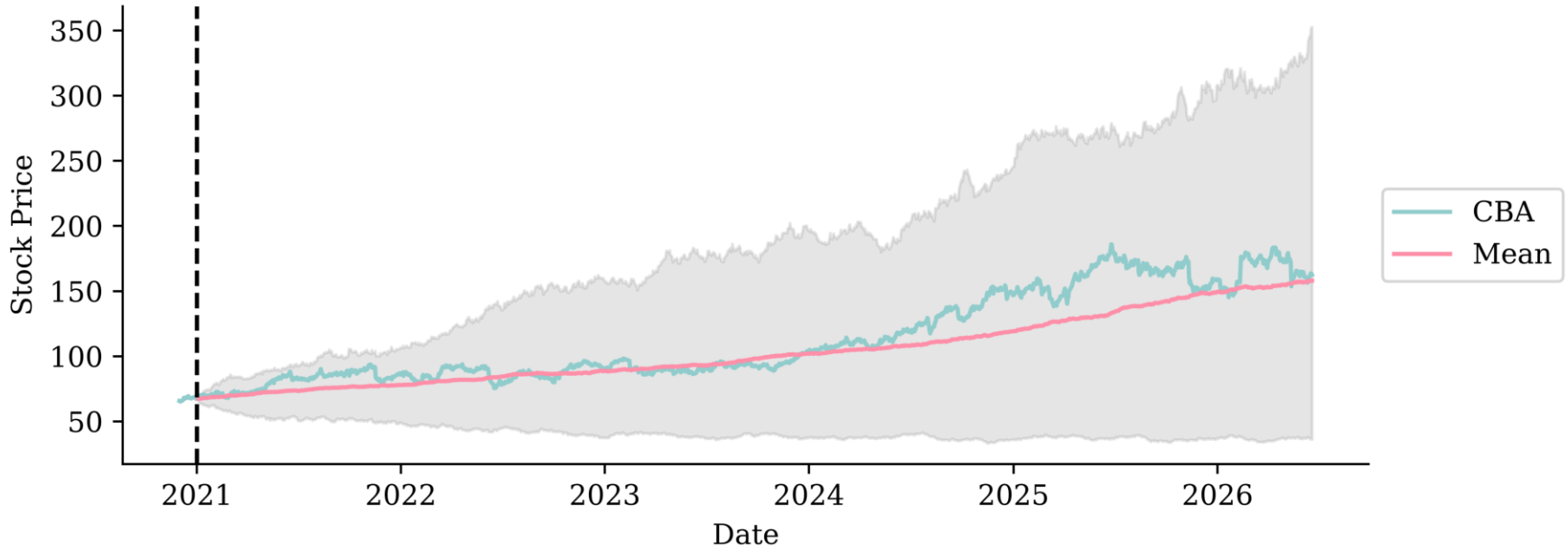
Many noisy forecasts

```
num_forecasts = 100
forecasts = []
for i in range(num_forecasts):
    sim = log_to_price(noisy_autoregressive_forecast(lr, X_test, sigma), last_price)
    forecasts.append(sim)
noisy_forecasts = pd.concat(forecasts, axis=1)
noisy_forecasts.index = X_test.index
```



95% “prediction intervals”

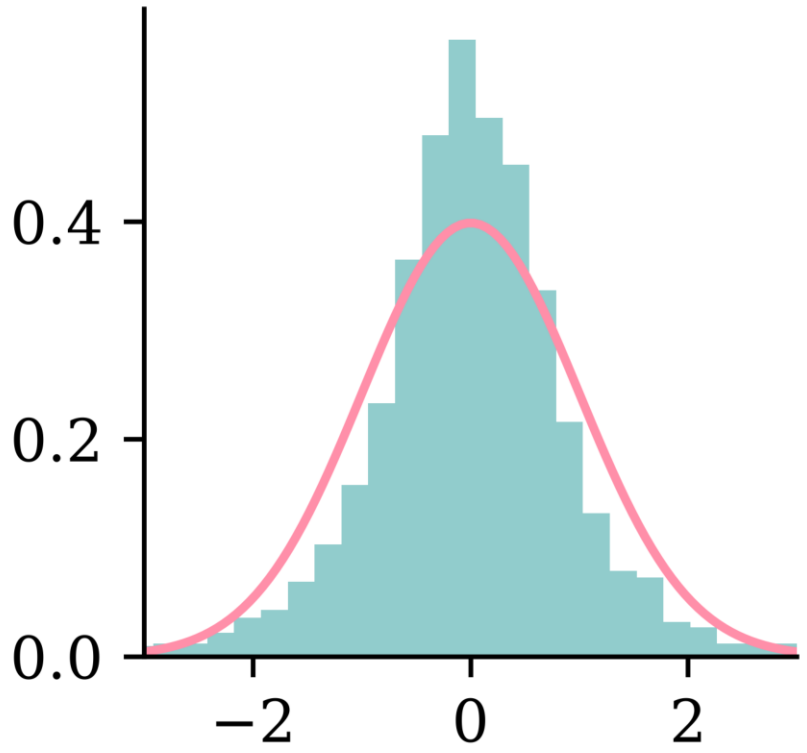
```
# Calculate quantiles for the forecasts
lower_quantile = noisy_forecasts.quantile(0.025, axis=1)
upper_quantile = noisy_forecasts.quantile(0.975, axis=1)
mean_forecast = noisy_forecasts.mean(axis=1)
```



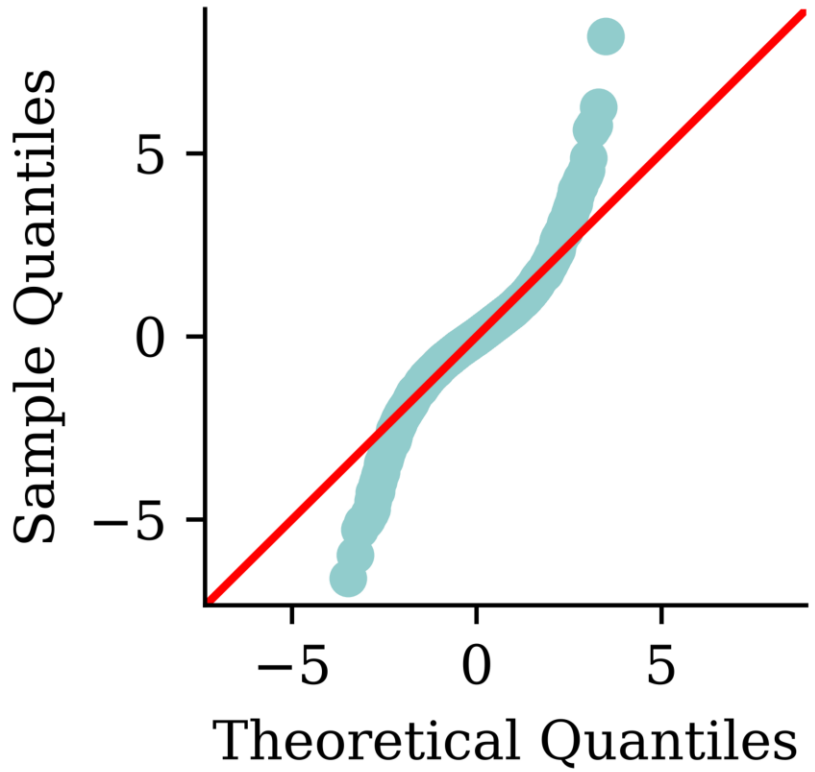
Residuals

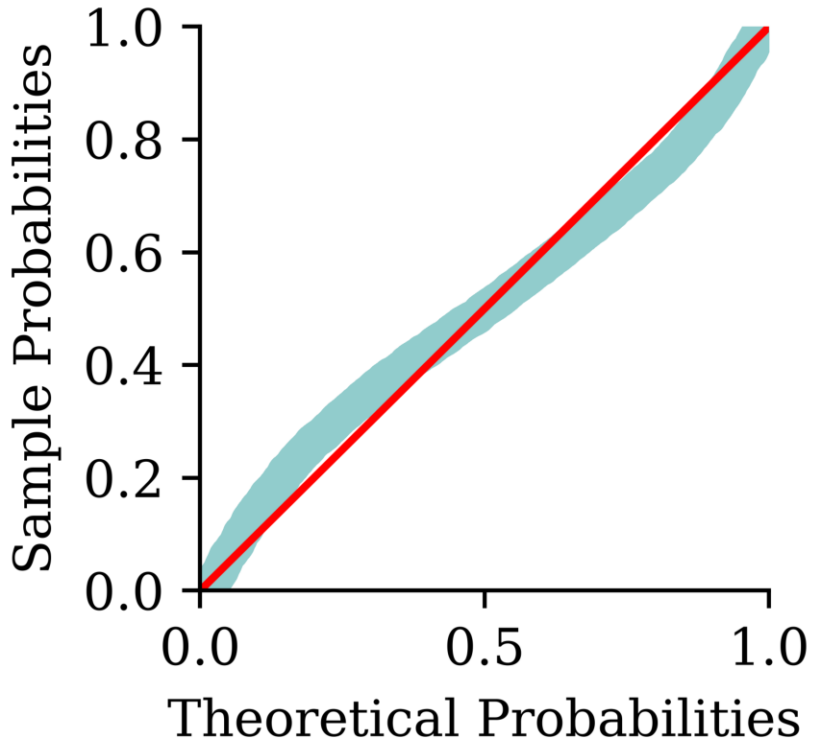
```
1 y_pred = lr.predict(X_train)
2 residuals = y_train - y_pred
3 residuals -= np.mean(residuals)
4 residuals /= np.std(residuals)
5 stats.shapiro(residuals)
```

```
ShapiroResult(statistic=np.float64(0.939218900(
pvalue=np.float64(6.991031400920509e-38))
```

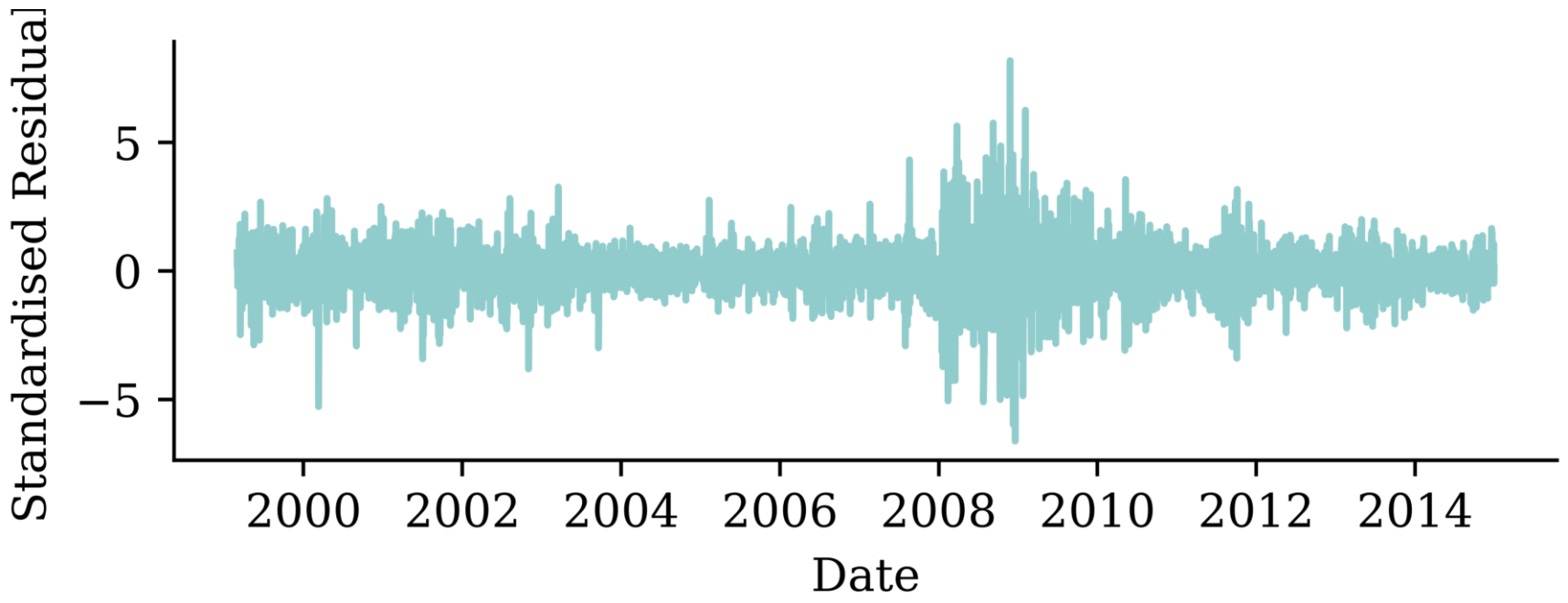


Q-Q plot and P-P plot



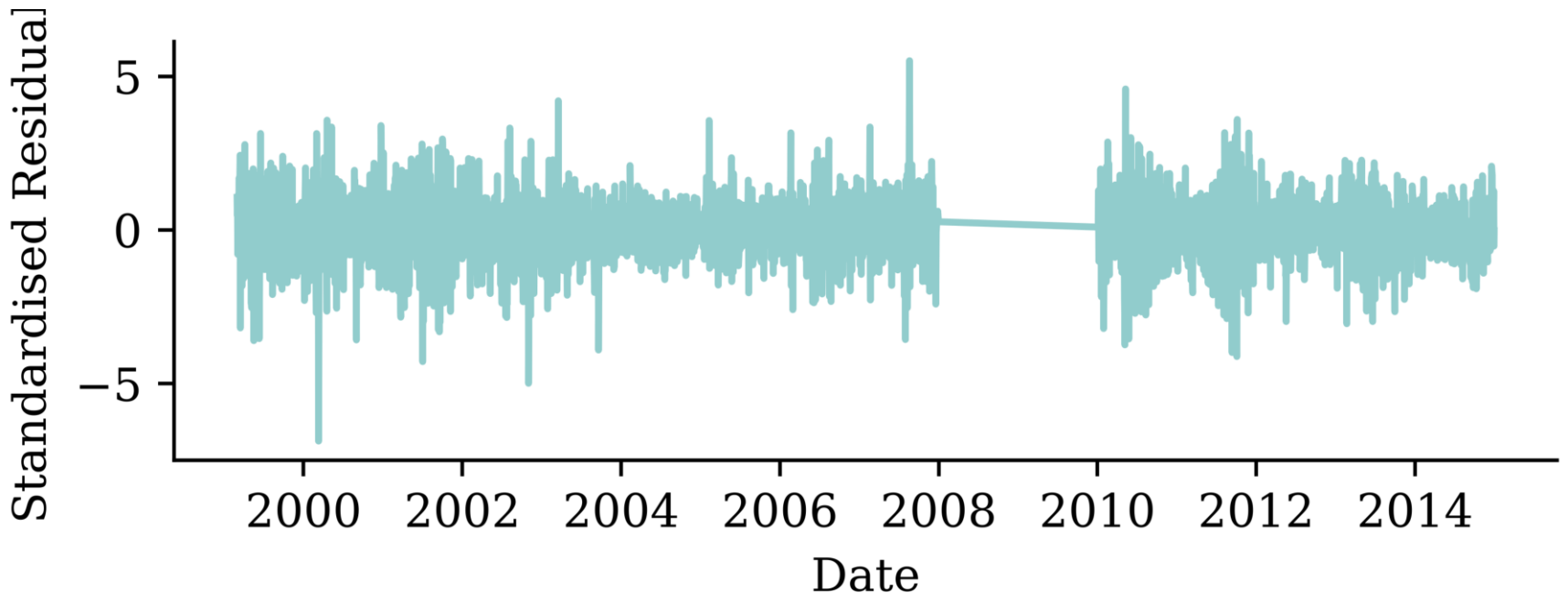


Residuals against time



Heteroskedasticity!

Retrain on less data

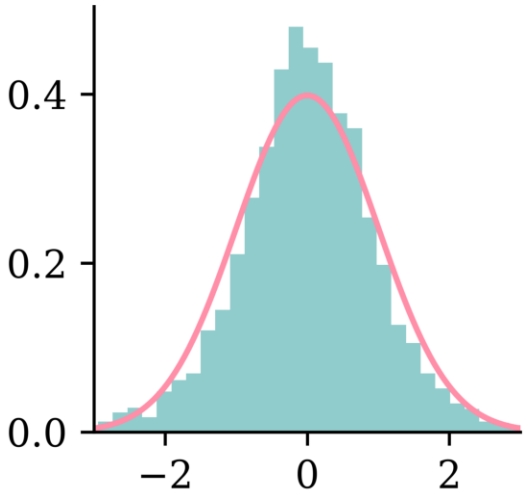


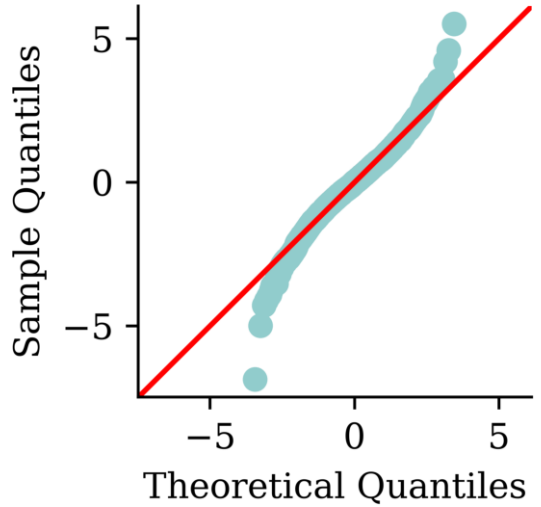
Refit the model without the GFC crisis years.

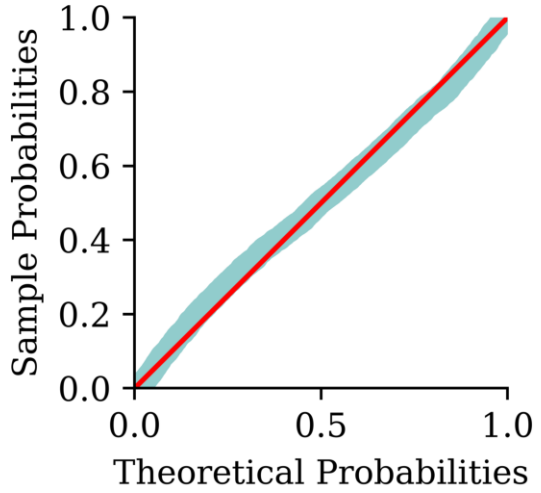
Residual diagnostics

```
1 stats.shapiro(res2)
```

```
ShapiroResult(statistic=np.float64(0.983029219714092), pvalue=np.float64(3.326265726133293e-20))
```







UNSW SYDNEY

Lecture Outline

- Introduction
- Traditional Regression
- Stochastic Forecasts
- **GLMs and Neural Networks**
- Combined Actuarial Neural Network
- Mean-Variance Estimation Network
- Mixture Density Network
- Distributional Refinement Network
- Uncertainty and Regularisation
- Dropout and Ensembles

French motor claim sizes

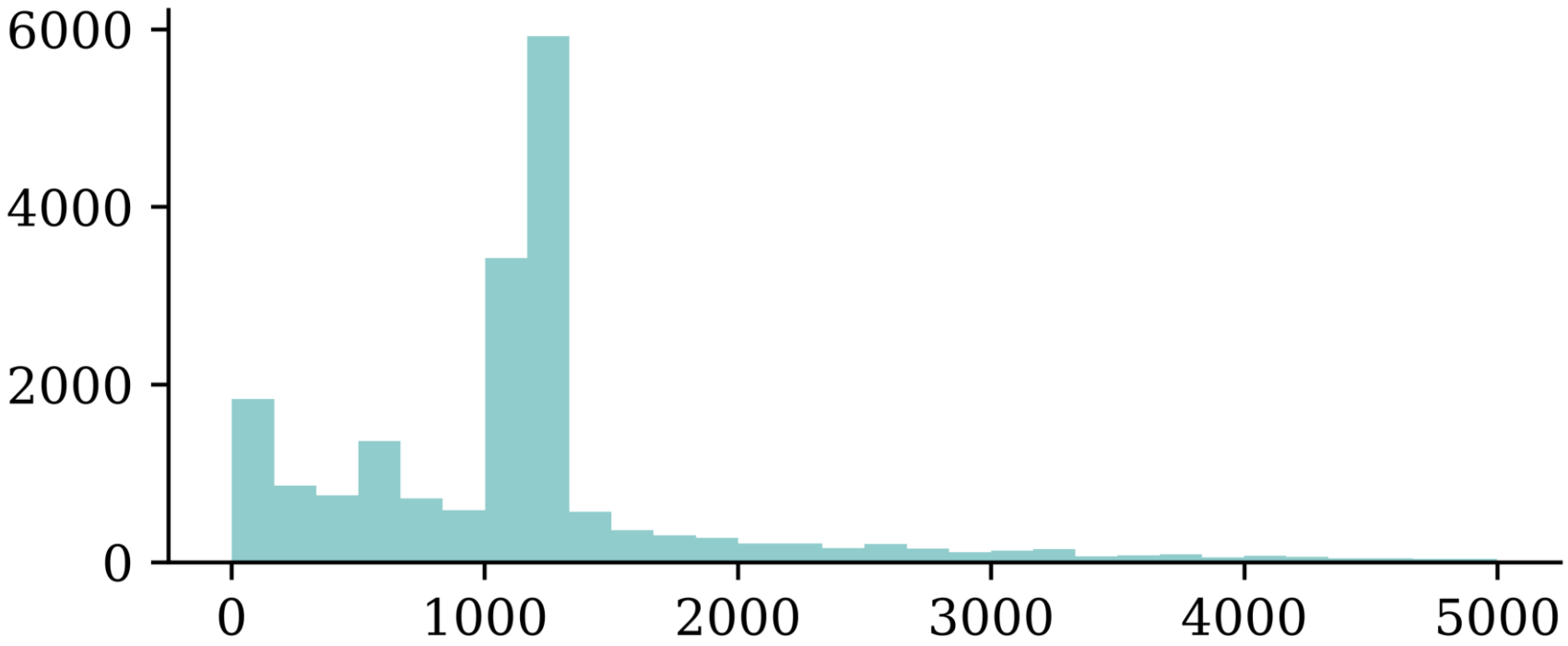
```
sev = pd.read_csv('data/freMTPL2sev.csv')
cov = pd.read_csv('data/freMTPL2freq.csv').drop(columns=['ClaimNb'])
sev = pd.merge(sev, cov, on='IDpol', how='left').drop(columns=["IDpol"]).dropna()
sev
```

	ClaimAmount	Exposure	VehPower	VehAge	DrivAge	BonusMalus	Veh
0	995.20	0.59	11.0	0.0	39.0	56.0	B12
1	1128.12	0.95	4.0	1.0	49.0	50.0	B12
...	...	...	...	...	...	...	...
26637	767.55	0.43	6.0	0.0	67.0	50.0	B2
26638	1500.00	0.28	7.0	2.0	36.0	60.0	B12

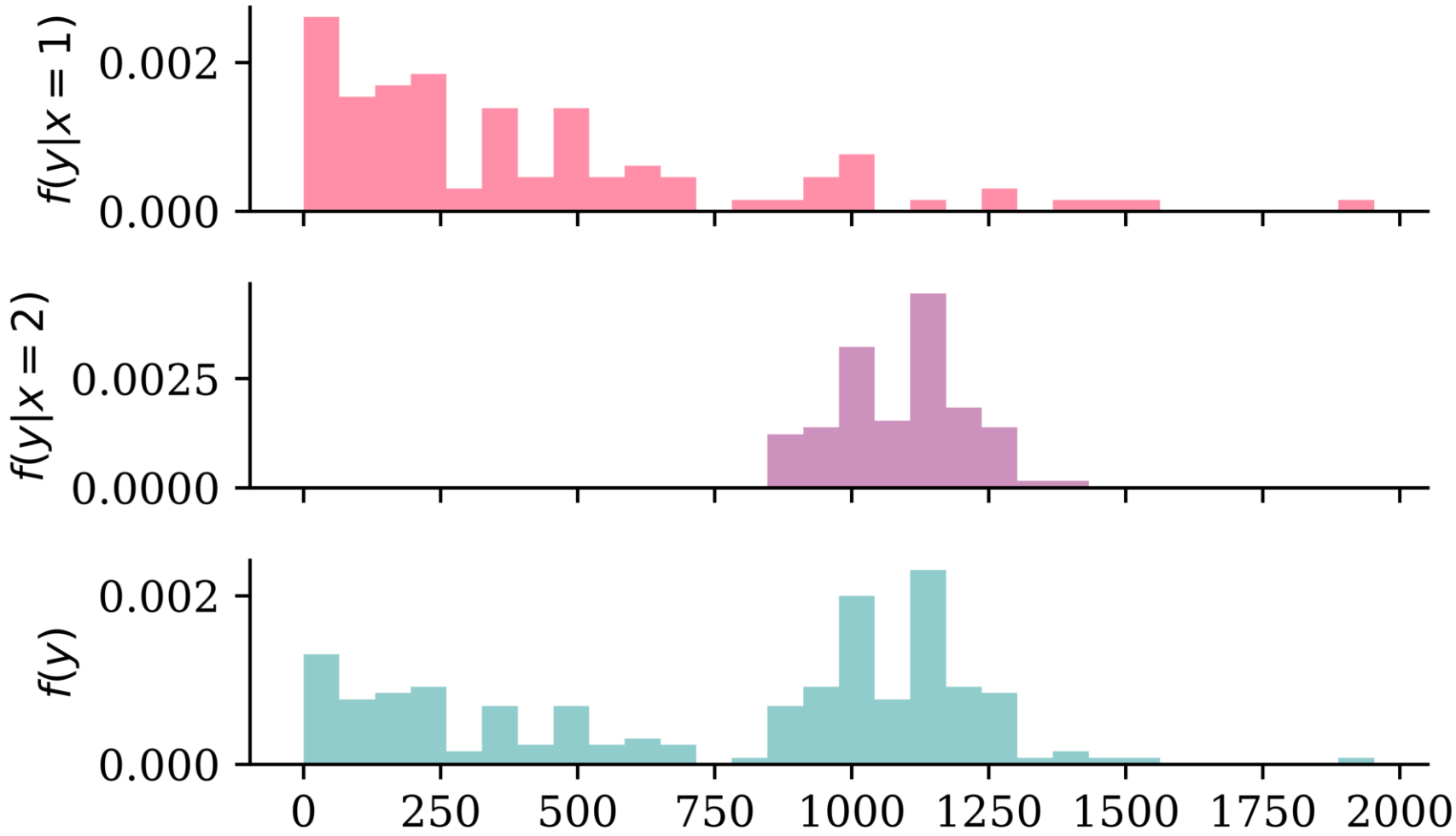
26444 rows × 11 columns

Preprocessing

```
X_train, X_test, y_train, y_test = train_test_split(
  sev.drop("ClaimAmount", axis=1), sev["ClaimAmount"], random_state=2023)
ct = make_column_transformer(
    (make_pipeline(OrdinalEncoder(), StandardScaler()), ["Area", "VehGas"]),
    ("drop", ["VehBrand", "Region"]), remainder=StandardScaler())
X_train = ct.fit_transform(X_train)
X_test = ct.transform(X_test)
plt.hist(y_train[y_train < 5000], bins=30);
```



Doesn't prove that $Y|\boldsymbol{X} = \boldsymbol{x}$ is multimodal



Gamma GLM

Suppose a fitted Gamma GLM model has

- a log link function $g(x) = \log(x)$ and
- regression coefficients $\boldsymbol{\beta} = (\beta_0, \beta_1, \beta_2, \beta_3)$.

Then, it estimates the conditional mean of Y given a new instance $\boldsymbol{x} = (1, x_1, x_2, x_3)$ as follows:

$$\mathbb{E}[Y|\boldsymbol{X} = \boldsymbol{x}] = g^{-1}(\langle \boldsymbol{\beta}, \boldsymbol{x} \rangle) = \exp\left(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3\right).$$

A GLM can model any other exponential family distribution using an appropriate link function g.

Gamma GLM loss

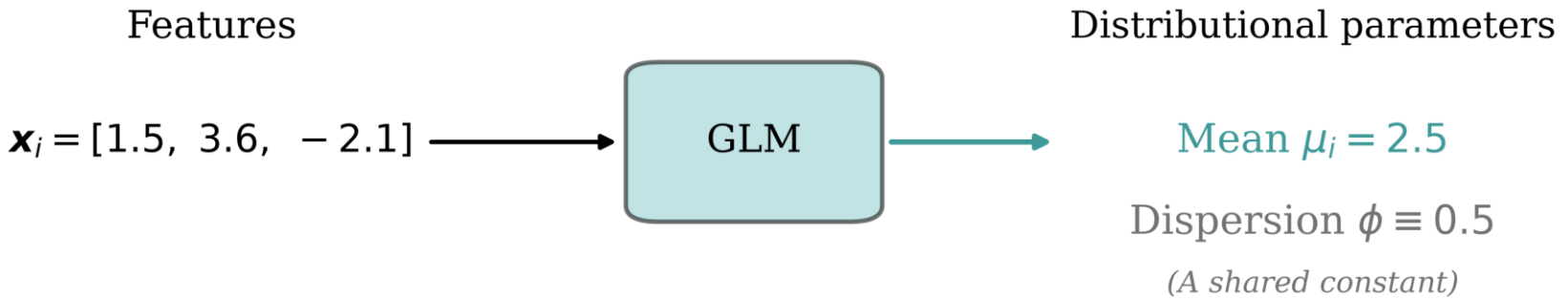
If $Y|\boldsymbol{X} = \boldsymbol{x}$ is a Gamma r.v. with mean $\mu(\boldsymbol{x}; \boldsymbol{\beta})$ and dispersion parameter ϕ, we can minimise the negative log-likelihood (NLL)

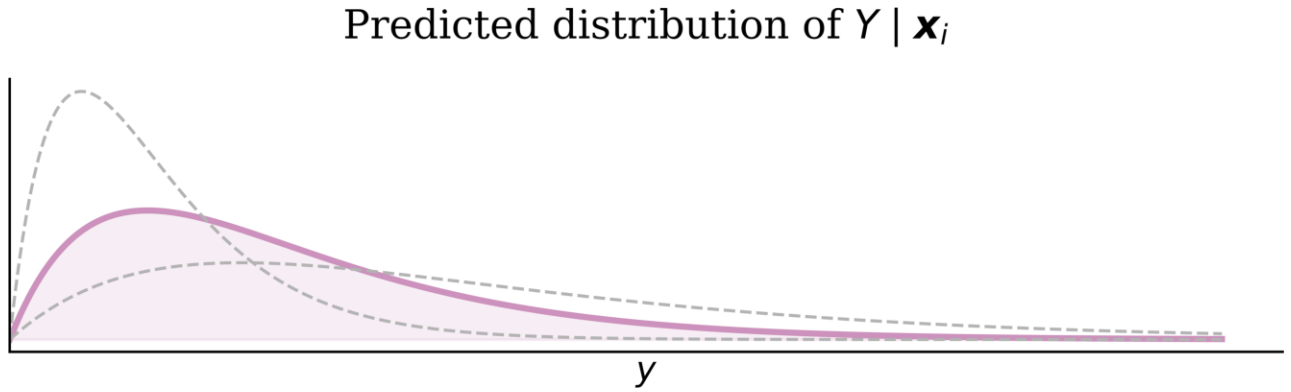
$$\text{NLL} \propto \sum_{i=1}^{n} \left[\log \mu(\boldsymbol{x}_i; \boldsymbol{\beta}) + \frac{y_i}{\mu(\boldsymbol{x}_i; \boldsymbol{\beta})} \right] + \text{const},$$

i.e., we ignore the dispersion parameter ϕ while estimating the regression coefficients.

What the GLM predicts

A GLM predicts the *mean* for each input; the dispersion is a single shared constant, estimated separately.





Fitting steps

Step 1. Use the advanced second derivative iterative method to find the regression coefficients:

$$\boldsymbol{\beta}^* = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} \left[\log \mu(\boldsymbol{x}_i; \boldsymbol{\beta}) + \frac{y_i}{\mu(\boldsymbol{x}_i; \boldsymbol{\beta})} \right]$$

Step 2. Estimate the dispersion parameter:

$$\phi = \frac{1}{n-p} \sum_{i=1}^{n} \frac{\left(y_i - \mu(\boldsymbol{x}_i; \boldsymbol{\beta}^*)\right)^2}{\mu(\boldsymbol{x}_i; \boldsymbol{\beta}^*)^2}$$

(Here, p is the number of coefficients in the model. If this p doesn't include the intercept, then the scaling should be $\frac{1}{n-(p+1)}$.)

Gamma GLM

In Python, we can fit a Gamma GLM as follows:

```python
# Add a column of ones to include an intercept in the model
X_train_design = sm.add_constant(X_train)

# Create a Gamma GLM with a log link function
gamma_glm = sm.GLM(y_train, X_train_design,
                   family=sm.families.Gamma(sm.families.links.Log()))

# Fit the model
gamma_glm = gamma_glm.fit()
```

```python
gamma_glm.params
```

```
const                     7.648131
pipeline__Area           -0.099472
                             ...
remainder__BonusMalus     0.157204
remainder__Density        0.010539
Length: 9, dtype: float64
```

```python
# Dispersion Parameter
mus = gamma_glm.predict(X_train_design)
residuals = y_train - mus
dof = (len(y_train)-X_train_design.shape[
phi_glm = np.sum(residuals**2/mus**2)/dof
print(phi_glm)
```

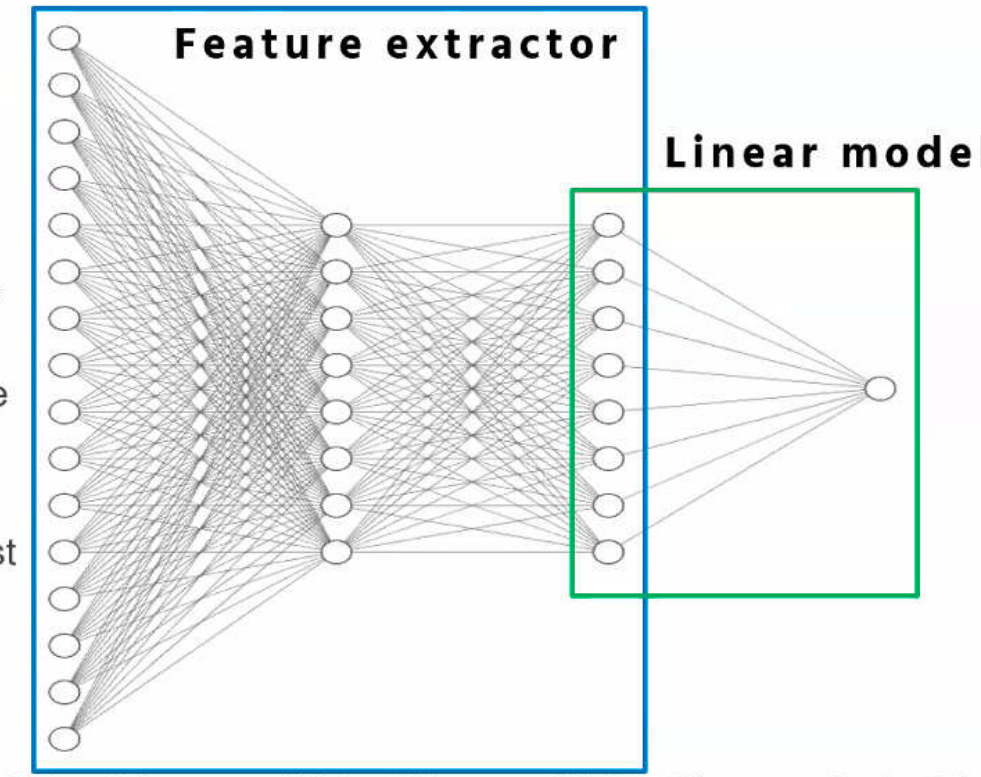
```
59.63363123736379
```

UNSW SYDNEY

ANN can feed into a GLM

FCN generalizes GLM

- Intermediate layers = representation learning, guided by supervised objective.
- Last layer = (generalized) linear model, where input variables = new representation of data
- No need to use GLM – strip off last layer and use learned features in, for example, XGBoost
- Or mix with traditional method of fitting GLM



Institute and Faculty of Actuaries

11 April 2022

14

Combining GLM & ANN.

Source: Ronald Richman (2022), *Mind the Gap — Safely Incorporating Deep Learning Models into the Actuarial Toolkit*, IFoA seminar, Slide 14.

UNSW SYDNEY

Deep GLM

A GLM has a *linear* predictor. A *deep GLM* (Tran et al., 2020) swaps that for a neural network — a non-linear mean — but keeps the *same Gamma loss*, so it still predicts a single Gamma distribution.

```
def gamma_loss(y_true, y_pred):
    return keras.ops.mean(keras.ops.log(y_pred) + y_true / y_pred)

random.seed(1)
inputs = Input(shape=X_train.shape[1:])
x = Dense(64, activation="leaky_relu")(inputs)
x = Dense(64, activation="leaky_relu")(x)
mu = Dense(1, activation="exponential")(x) + 1e-6
deep_glm = Model(inputs, mu)
deep_glm.compile(optimizer="adam", loss=gamma_loss)
deep_glm.fit(X_train, y_train, epochs=100, batch_size=64, verbose=0,
    callbacks=[EarlyStopping(patience=10, restore_best_weights=True)],
    validation_split=0.2);
```

Lecture Outline

- Introduction
- Traditional Regression
- Stochastic Forecasts
- GLMs and Neural Networks
- **Combined Actuarial Neural Network**
- Mean-Variance Estimation Network
- Mixture Density Network
- Distributional Refinement Network
- Uncertainty and Regularisation
- Dropout and Ensembles

CANN

The Combined Actuarial Neural Network is an actuarial neural network architecture proposed by Schelldorfer & Wüthrich (2019). We summarise the CANN approach as follows:

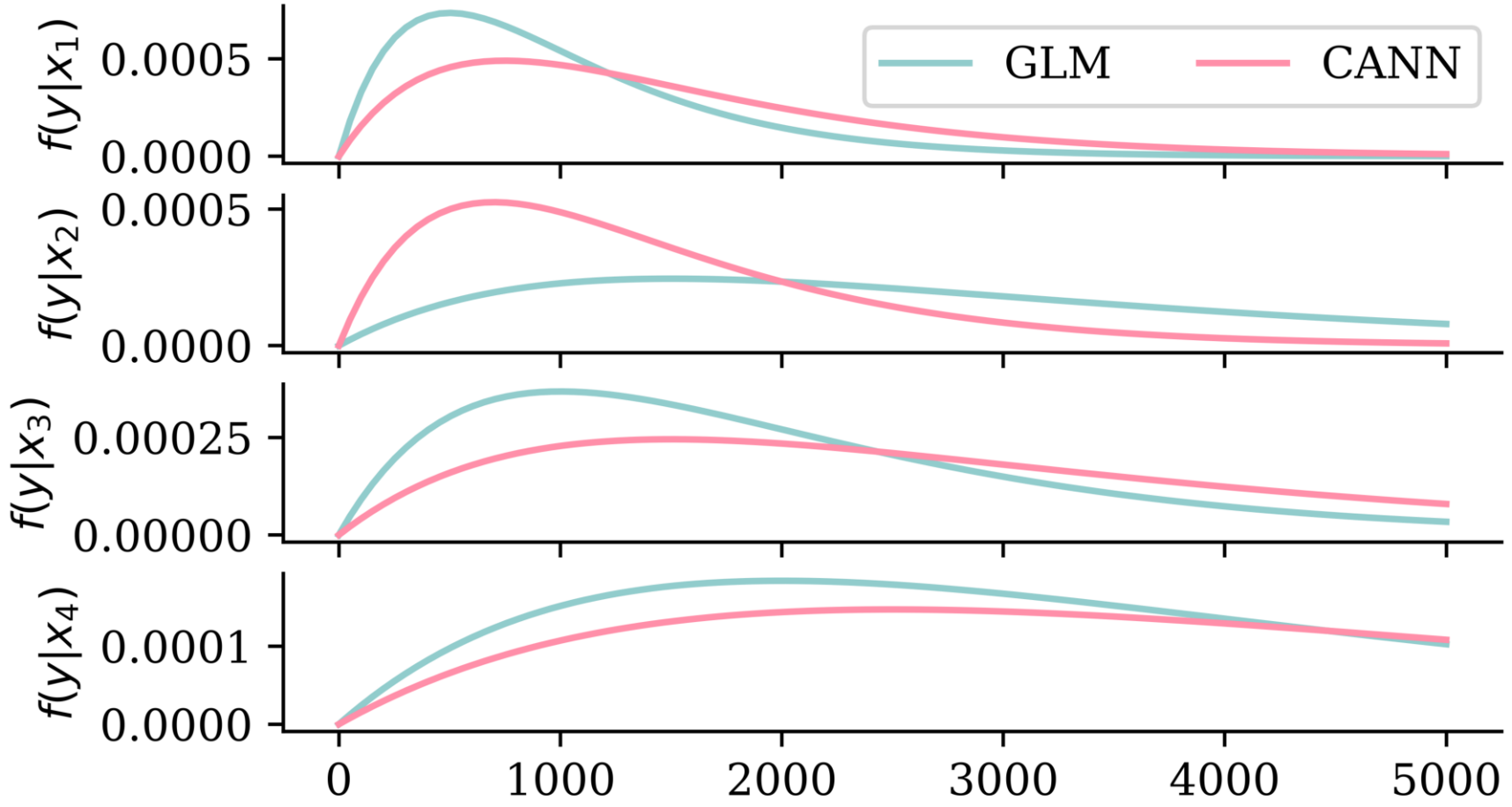
- Find coefficients $\boldsymbol{\beta}$ of the GLM with a link function $g(\cdot)$.
- Find weights $\boldsymbol{w} = (\boldsymbol{w}^{(1)}, \ldots, \boldsymbol{w}^{(K+1)})$ of a neural network $\boldsymbol{s} : \mathbb{R}^p \to \mathbb{R}$.
- Given a new instance $\boldsymbol{x}$, we have

$$\mathbb{E}[Y|\boldsymbol{X} = \boldsymbol{x}] = g^{-1}\Big(\langle \boldsymbol{\beta}, \boldsymbol{x} \rangle + s(\boldsymbol{x}; \boldsymbol{w})\Big).$$

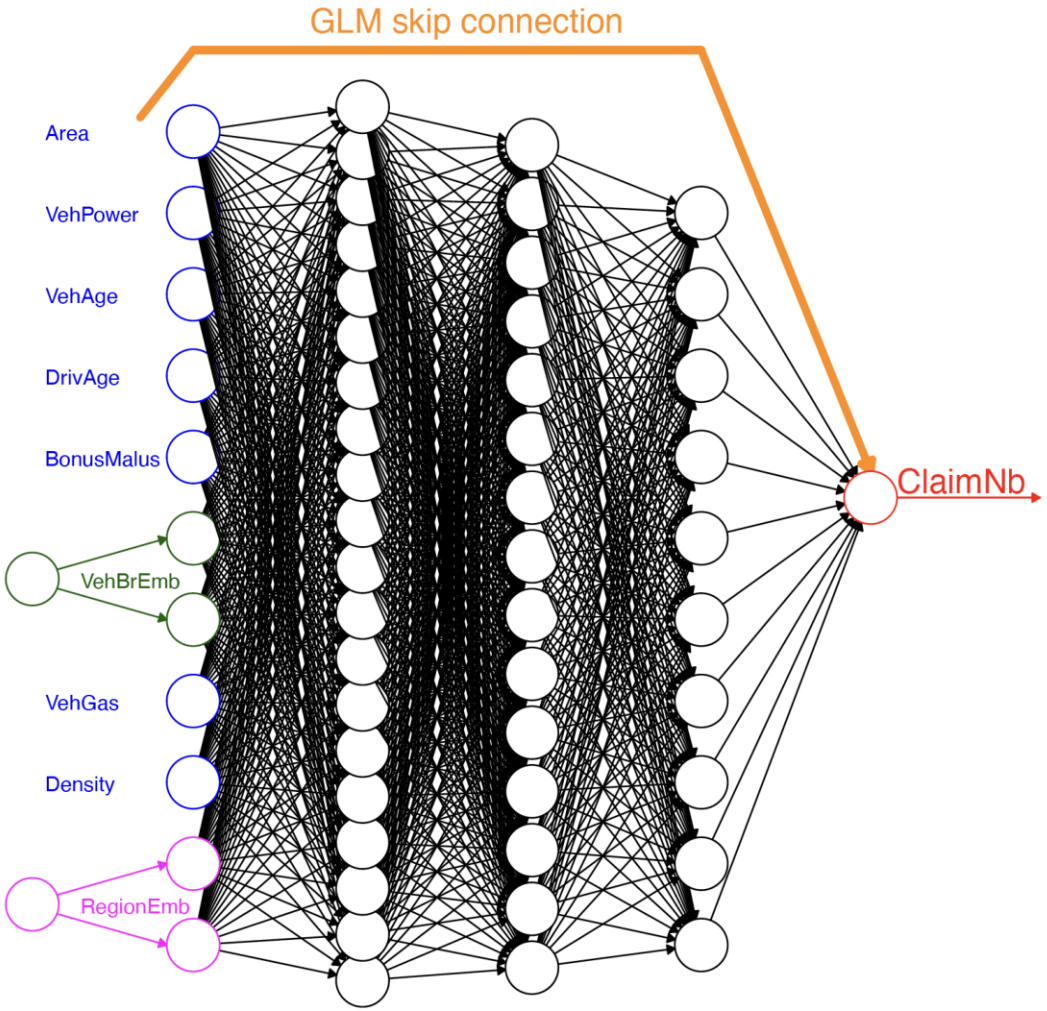
If just a sequential dense network, then

$$\mathbb{E}[Y|\boldsymbol{X} = \boldsymbol{x}] = g^{-1}\Big(\langle \boldsymbol{\beta}, \boldsymbol{x} \rangle + \Big\langle \boldsymbol{w}^{(K+1)}, \big(\boldsymbol{z}^{(K)} \circ \cdots \circ \boldsymbol{z}^{(1)}\big)(\boldsymbol{x}) \Big\rangle\Big).$$

Shifting the predicted distributions



Architecture



The CANN architecture.

Source: Figure 8 of Schelldorfer & Wüthrich (2019).

CANN architecture

```
random.seed(1)
inputs = Input(shape=X_train.shape[1:])

# GLM part (won't be updated during training)
glm_weights = gamma_glm.params.iloc[1:].values.reshape((-1, 1))
glm_bias = gamma_glm.params.iloc[0]
glm_part = Dense(1, activation='linear', trainable=False,
                 kernel_initializer=Constant(glm_weights),
                 bias_initializer=Constant(glm_bias))(inputs)

# Neural network part
x = Dense(64, activation='leaky_relu')(inputs)
nn_part = Dense(1, activation='linear',
                kernel_initializer="zeros",
                bias_initializer="zeros")(x)

# Combine GLM and CANN estimates
mu = keras.ops.exp(glm_part + nn_part) + 1e-6
cann = Model(inputs, mu)
```

Since this CANN predicts Gamma distributions, we reuse the same `gamma_loss` from the deep GLM.

Training the CANN

```
cann.compile(optimizer="adam", loss=gamma_loss)
hist = cann.fit(X_train, y_train,
    epochs=100,
    callbacks=[EarlyStopping(patience=10, restore_best_weights=True)],
    verbose=0,
    batch_size=64,
    validation_split=0.2)
```

Find the dispersion parameter.

```
mus = cann.predict(X_train, verbose=0).flatten()
residuals = y_train - mus
dof = (len(y_train)-(X_train.shape[1] + 1))
phi_cann = np.sum(residuals**2/mus**2) / dof
print(phi_cann)
```

```
64.48475298524968
```

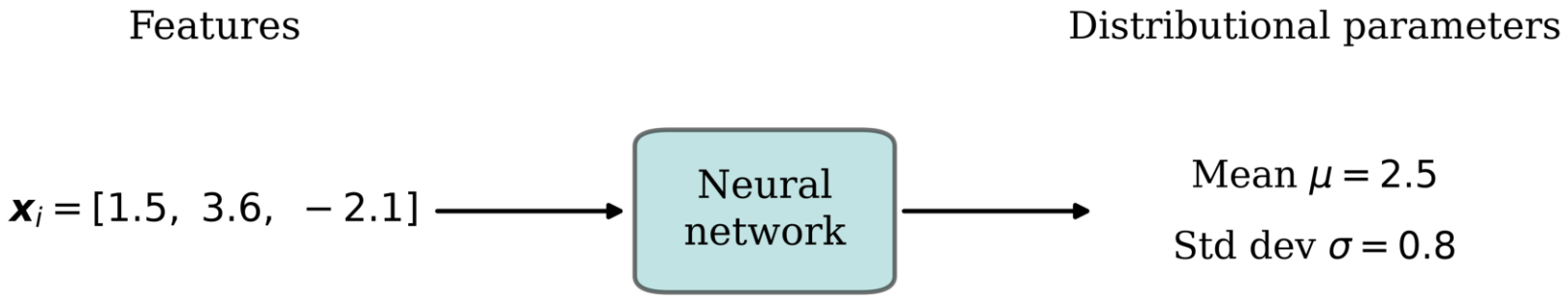
Lecture Outline

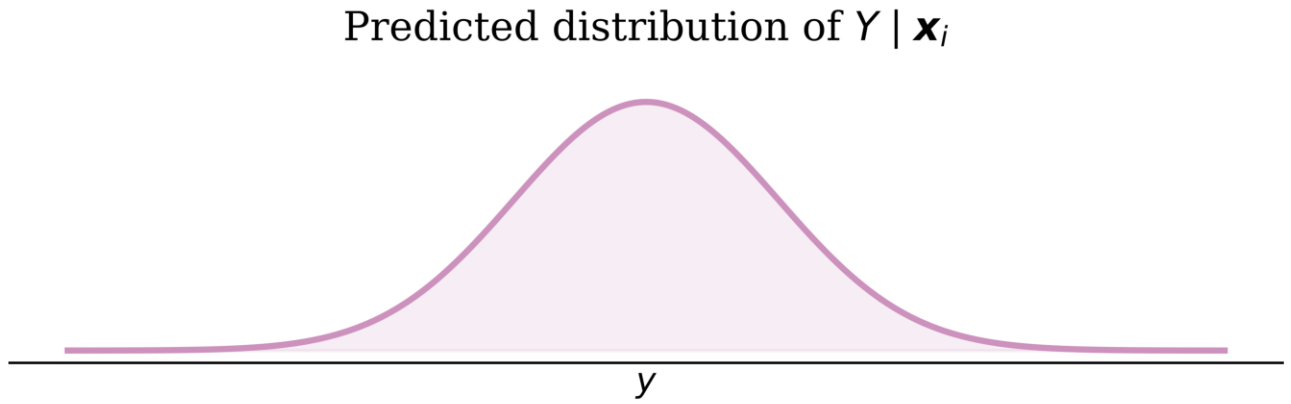
- Introduction
- Traditional Regression
- Stochastic Forecasts
- GLMs and Neural Networks
- Combined Actuarial Neural Network
- **Mean-Variance Estimation Network**
- Mixture Density Network
- Distributional Refinement Network
- Uncertainty and Regularisation
- Dropout and Ensembles

Generating normal distribution parameters

Our model is

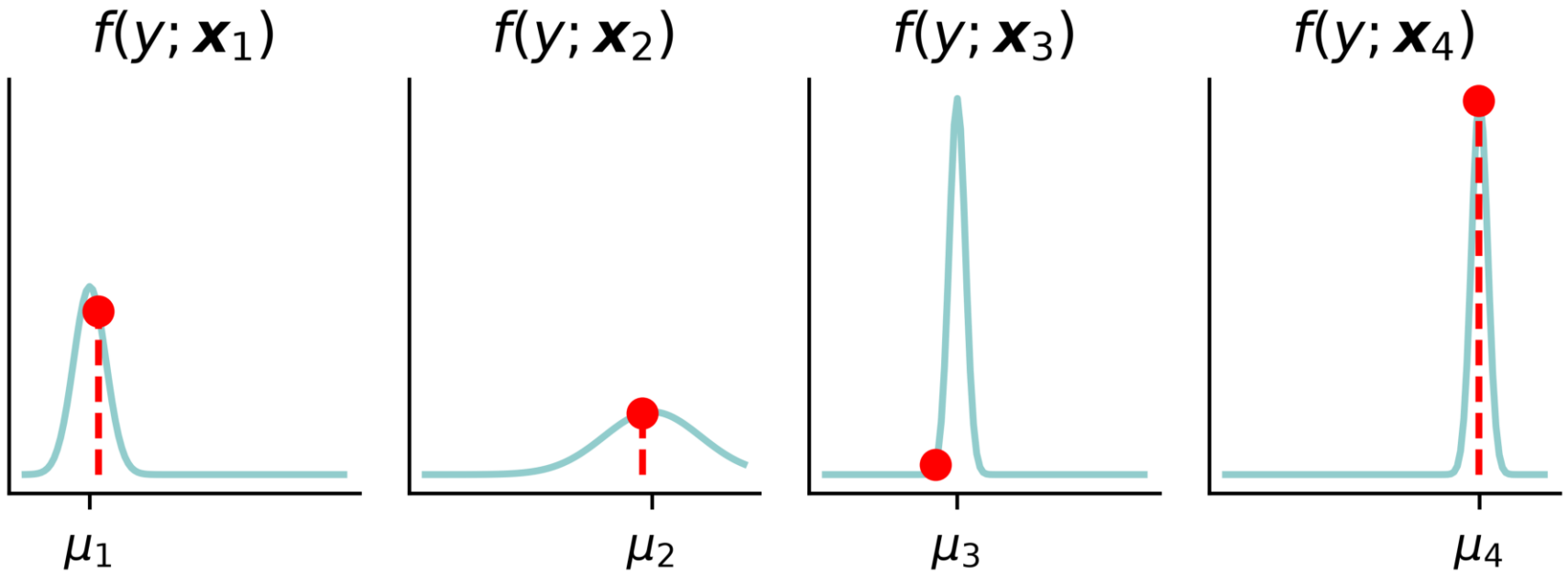
$$Y \mid \boldsymbol{x} \sim \mathcal{N}\big(\mu(\boldsymbol{x}), \sigma^2(\boldsymbol{x})\big).$$





In-class exercise

Make a neural network that models $Y \mid \boldsymbol{x} \sim \mathcal{N}\big(\mu(\boldsymbol{x}), \sigma^2(\boldsymbol{x})\big)$.



Task: Assume you have `X_train` and `y_train` loaded and write the following code, everything up to the `model.fit(X_train, y_train)` line.

Mean-variance estimation networks: Nix & Weigend (1994).

Lecture Outline

- Introduction
- Traditional Regression
- Stochastic Forecasts
- GLMs and Neural Networks
- Combined Actuarial Neural Network
- Mean-Variance Estimation Network
- **Mixture Density Network**
- Distributional Refinement Network
- Uncertainty and Regularisation
- Dropout and Ensembles

Mixture density network

The mean-variance network outputs the parameters of a *single* normal. But our data may be *multimodal*, or have some other shape which no single normal or gamma can capture.

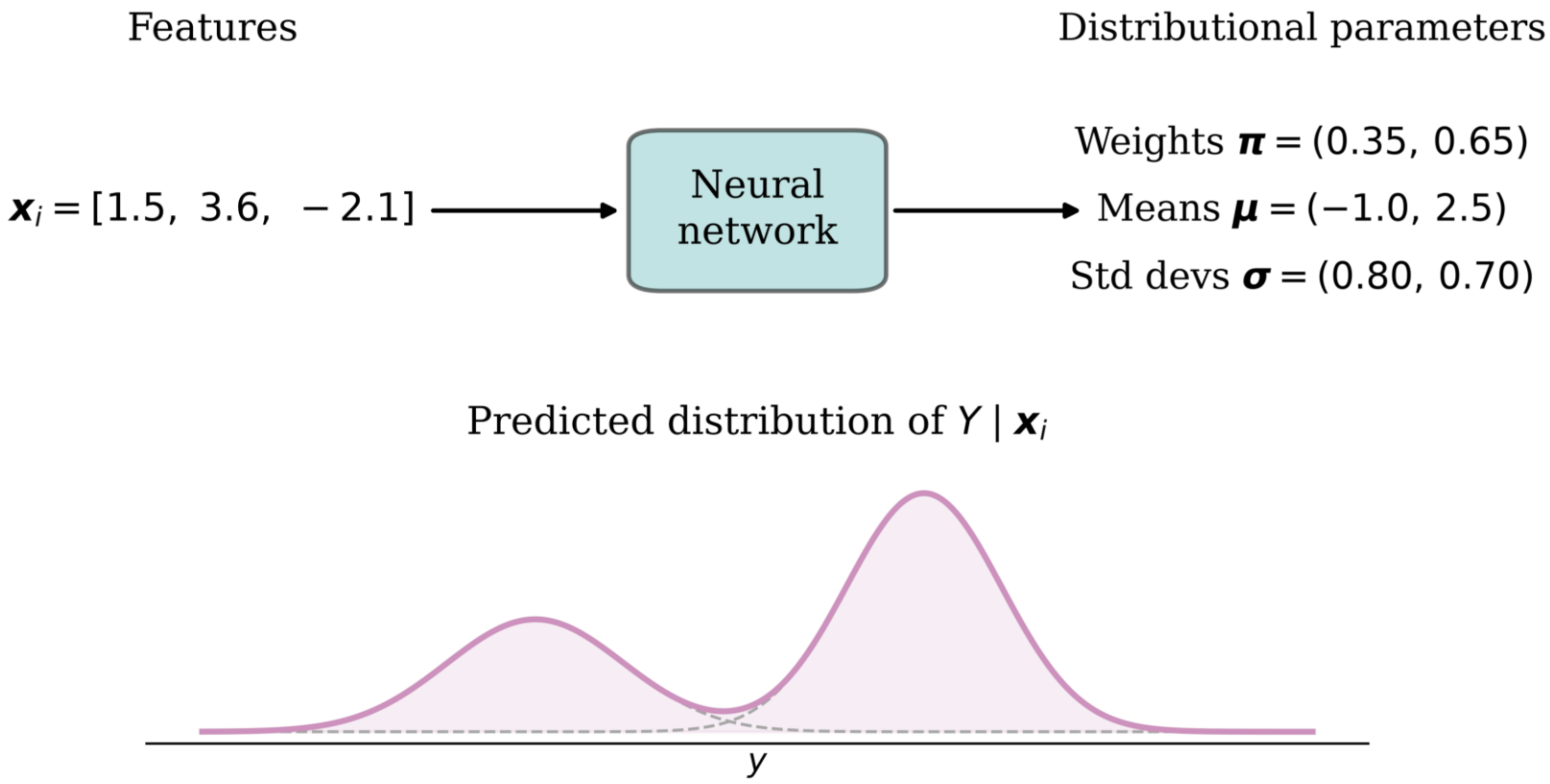
A *mixture density network* (MDN) operates the same way, except now we model $Y \mid \boldsymbol{x}$ as a *mixture* of K components. The network outputs the mixing weights $\boldsymbol{\pi}(\boldsymbol{x})$ and each component's parameters, giving the density

$$f_{Y|\boldsymbol{X}}(y \mid \boldsymbol{x}) = \sum_{k=1}^{K} \pi_k(\boldsymbol{x})\, f_k(y \mid \boldsymbol{x}), \qquad \pi_k(\boldsymbol{x}) \geq 0, \ \sum_{k=1}^{K} \pi_k(\boldsymbol{x}) = 1.$$

Each component f_k can be any parametric family (normal, gamma, etc.).

Mixture density networks: Bishop (1994).

A two-component normal MDN



A two-component Gamma mixture

Suppose there are two types of claims:

- Type I: $Y_1|\boldsymbol{X} = \boldsymbol{x} \sim \text{Gamma}(\alpha_1(\boldsymbol{x}), \beta_1(\boldsymbol{x}))$ and,
- Type II: $Y_2|\boldsymbol{X} = \boldsymbol{x} \sim \text{Gamma}(\alpha_2(\boldsymbol{x}), \beta_2(\boldsymbol{x}))$.

(Note the new gamma parametrisation.) So the claim amount $Y|\boldsymbol{X} = \boldsymbol{x}$ has density

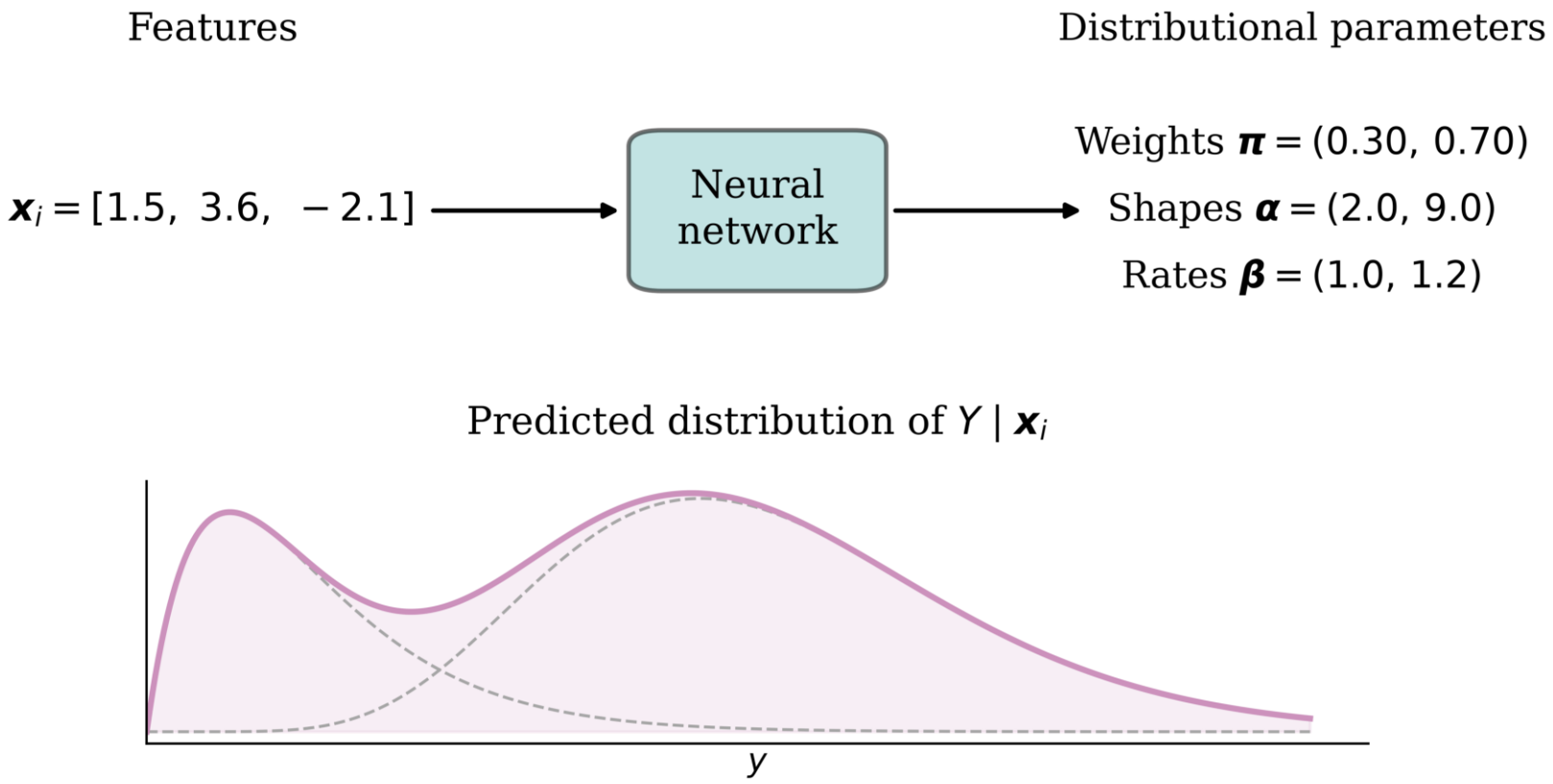
$$f_{Y|\boldsymbol{X}}(y|\boldsymbol{x}) = \pi_1(\boldsymbol{x}) \cdot \frac{\beta_1(\boldsymbol{x})^{\alpha_1(\boldsymbol{x})}}{\Gamma(\alpha_1(\boldsymbol{x}))} \mathrm{e}^{-\beta_1(\boldsymbol{x})y} y^{\alpha_1(\boldsymbol{x})-1} + \pi_2(\boldsymbol{x}) \cdot \frac{\beta_2(\boldsymbol{x})^{\alpha_2(\boldsymbol{x})}}{\Gamma(\alpha_2(\boldsymbol{x}))} \mathrm{e}^{-\beta_2(\boldsymbol{x})y} y^{\alpha_2(\boldsymbol{x})-1},$$

where $\pi_1(\boldsymbol{x})$ is the probability of a Type I claim. So here the network's six outputs are the mixing weights, shapes and rates:

$$\text{MDN}(\boldsymbol{x}; \boldsymbol{w}^*) = \big(\pi_1, \pi_2,\ \alpha_1, \alpha_2,\ \beta_1, \beta_2\big).$$

Gamma MDNs for insurance claim amounts: Delong et al. (2021); MDNs for loss reserving: Al-Mudafer et al. (2022).

The Gamma MDN



Gamma MDN architecture

The following code implements the gamma MDN from the previous slide.

```
random.seed(1)

n_components = 2
inputs = Input(shape=X_train.shape[1:])

x = Dense(64, activation="relu")(inputs)
x = Dense(64, activation="relu")(x)

pis = Dense(n_components, activation="softmax")(x)
alphas = Dense(n_components, activation="softplus")(x) + 1e-6
betas = Dense(n_components, activation="softplus")(x) + 1e-6

outputs = Concatenate()([pis, alphas, betas])
gamma_mdn = Model(inputs, outputs)
```

Loss & training

`torch.distributions` to the rescue.

```
def gamma_mixture_nll(y_true, y_pred):
    pis = y_pred[:, :n_components]
    alphas = y_pred[:, n_components:2*n_components]
    betas = y_pred[:, 2*n_components:]
    mixture = MixtureSameFamily(
        mixture_distribution=Categorical(probs=pis),
        component_distribution=Gamma(concentration=alphas, rate=betas))
    return -keras.ops.mean(mixture.log_prob(keras.ops.squeeze(y_true)))
```

We can now train it like any other Keras model.

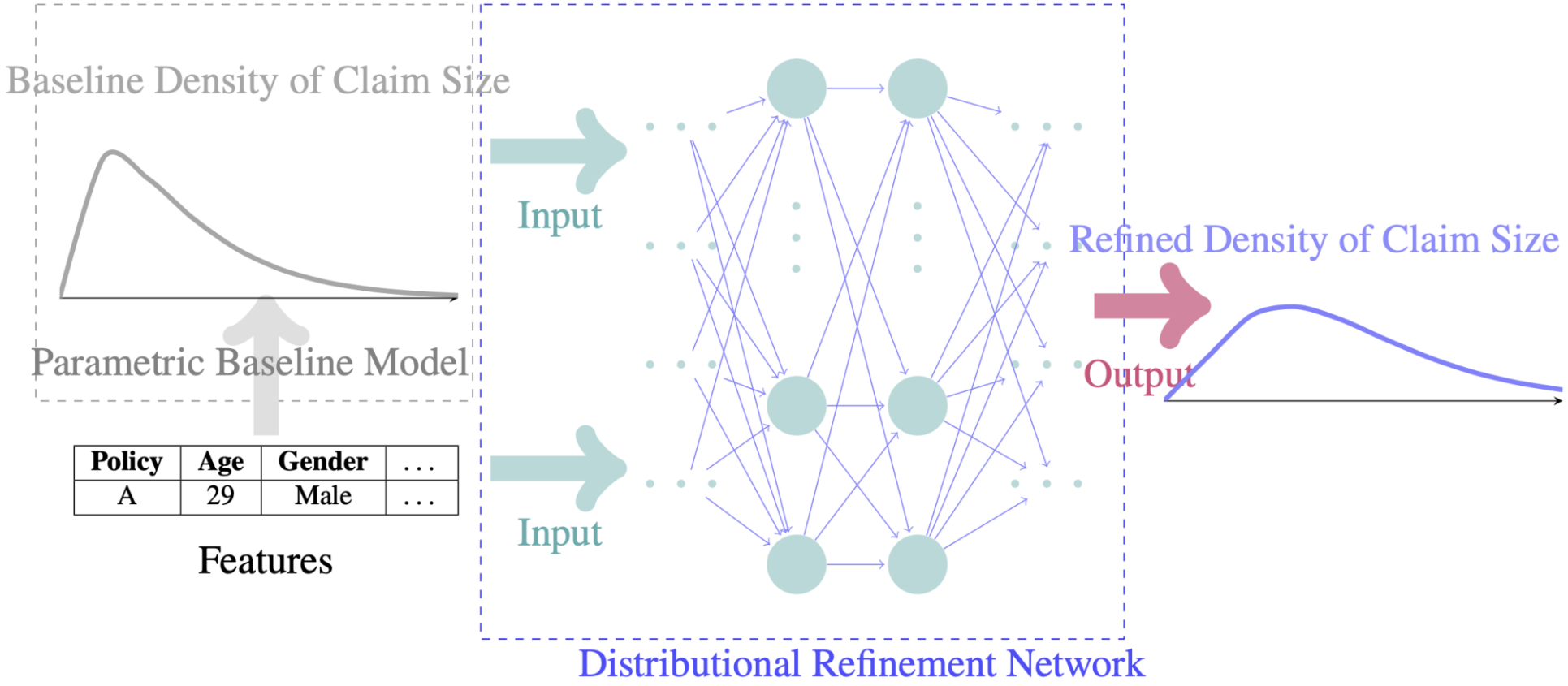
```
gamma_mdn.compile(optimizer="adam", loss=gamma_mixture_nll)
gamma_mdn.fit(X_train, y_train,
    epochs=100,
    callbacks=[EarlyStopping(patience=10, restore_best_weights=True)],
    verbose=0,
    batch_size=64,
    validation_split=0.2)
```

Lecture Outline

- Introduction
- Traditional Regression
- Stochastic Forecasts
- GLMs and Neural Networks
- Combined Actuarial Neural Network
- Mean-Variance Estimation Network
- Mixture Density Network
- **Distributional Refinement Network**
- Uncertainty and Regularisation
- Dropout and Ensembles

The DRN architecture

Start from a *trusted* parametric baseline (e.g. a GLM), then let a neural network make *small adjustments* to the whole distribution.

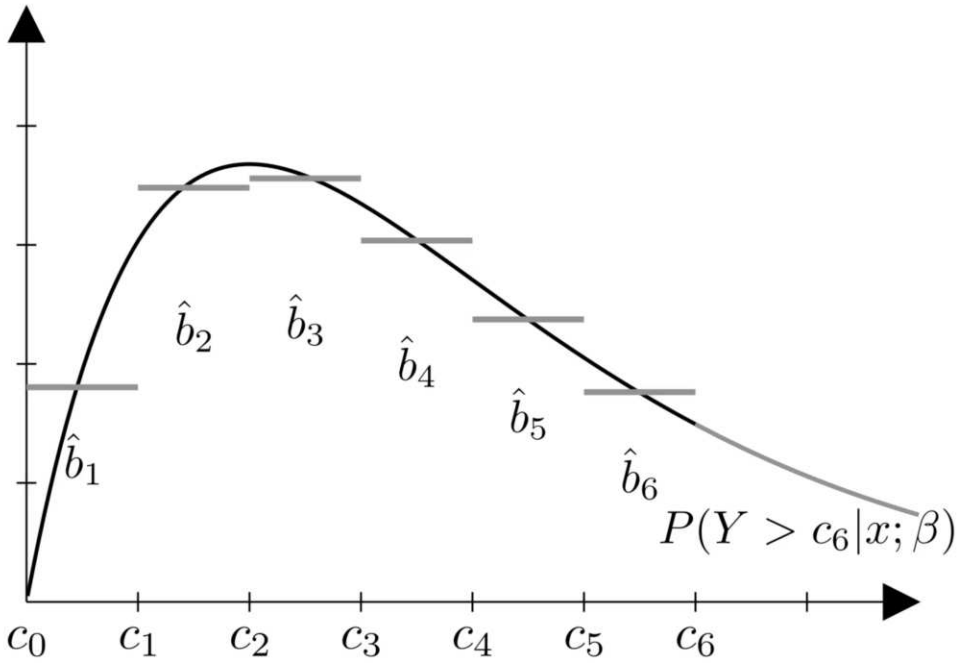


The DRN keeps the baseline's shape but flexibly *refines* it using the features.

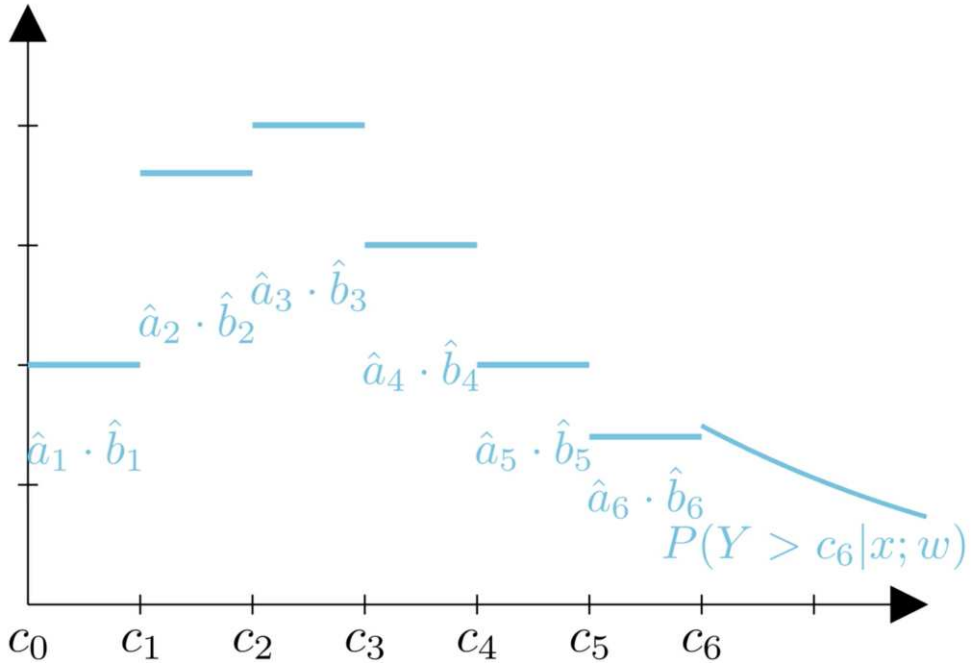
Source: Avanzi et al. (2026).

Refining the distribution

Discretise the baseline into bins, then multiply each bin's mass by a learned adjustment factor $\hat{a}_k$.



Baseline: discretised masses $\hat{b}_k$.



Refined: adjusted masses $\hat{a}_k \cdot \hat{b}_k$.

A penalty keeps the adjustments *faithful* to the baseline, so the model stays interpretable and trustworthy.

Source: Avanzi et al. (2026).

Implemented in the `drn` package

The `drn` package fits the DRN architecture along with today's earlier models:

```python
from drn import GLM, CANN, MDN, DRN, nll

glm  = GLM("gamma").fit(X_train, y_train)
cann = CANN(glm).fit(X_train, y_train)
mdn  = MDN("gamma", num_components=2).fit(X_train, y_train)
drn  = DRN(glm).fit(X_train, y_train)
```

```python
for name, model in {"GLM": glm, "CANN": cann, "MDN": mdn, "DRN": drn}.items():
    print(f"{name}: {nll(model.predict(X_test), y_test):.2f}")
```

```
GLM: 11.02
CANN: 10.27
MDN: 8.78
DRN: 8.32
```

Source: Avanzi et al. (2026).

Lecture Outline

- Introduction
- Traditional Regression
- Stochastic Forecasts
- GLMs and Neural Networks
- Combined Actuarial Neural Network
- Mean-Variance Estimation Network
- Mixture Density Network
- Distributional Refinement Network
- **Uncertainty and Regularisation**
- Dropout and Ensembles

Categories of uncertainty

There are two major categories of uncertainty in statistical or machine learning:

- Aleatoric uncertainty: the inherent variability associated with the data generating process.
- Epistemic uncertainty: the lack of knowledge, limited data information, parameter errors and model errors.

Sources of uncertainty

If you decide to predict the claim amount of an individual using a deep learning model, which source(s) of uncertainty are you dealing with?

1. The inherent variability of the data-generating process → aleatoric uncertainty.
2. Parameter error → epistemic uncertainty.
3. Model error → epistemic uncertainty.
4. Data uncertainty → epistemic uncertainty.

Traditional regularisation

Say all the m weights (excluding biases) are in the vector $\boldsymbol{\theta}$. If we change the loss function to

$$\text{Loss}_{1:n} = \frac{1}{n}\sum_{i=1}^{n}\text{Loss}_i + \lambda\sum_{j=1}^{m}|\theta_j|$$

this would be using L^1 regularisation. A loss like

$$\text{Loss}_{1:n} = \frac{1}{n}\sum_{i=1}^{n}\text{Loss}_i + \lambda\sum_{j=1}^{m}\theta_j^2$$

is called L^2 regularisation.

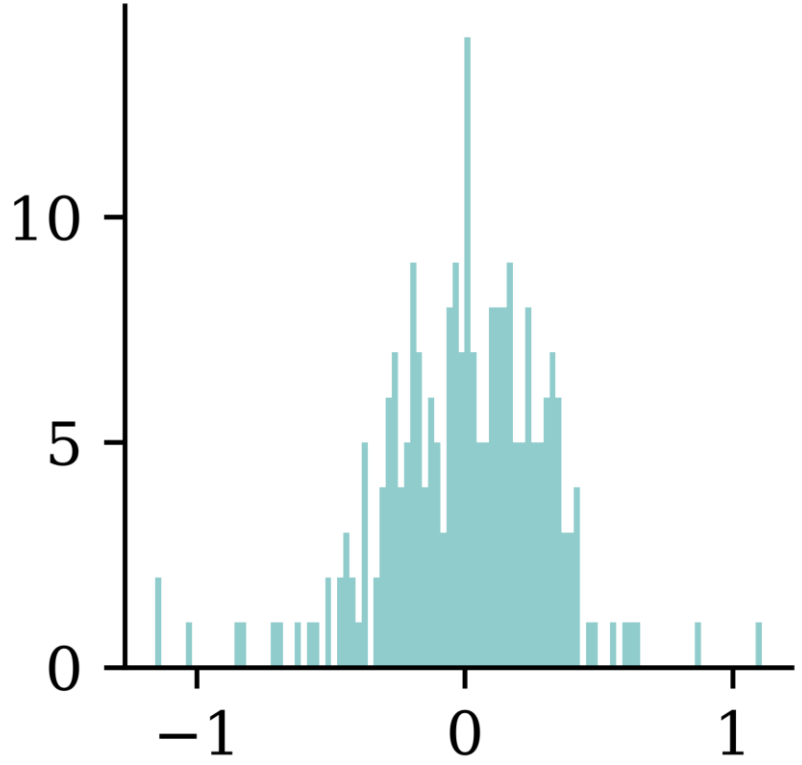
Regularisation in Keras

```
def l1_model(regulariser_strength=0.01):
  random.seed(123)
  model = Sequential([
      Dense(30, activation="leaky_relu",
        kernel_regularizer=L1(regulariser_strength)),
      Dense(1, activation="exponential")
  ])

  model.compile("adam", "mse")
  model.fit(X_train_sc, y_train, epochs=4, verbose=0)
  return model

def l2_model(regulariser_strength=0.01):
  random.seed(123)
  model = Sequential([
      Dense(30, activation="leaky_relu",
        kernel_regularizer=L2(regulariser_strength)),
      Dense(1, activation="exponential")
  ])

  model.compile("adam", "mse")
  model.fit(X_train_sc, y_train, epochs=10, verbose=0)
  return model
```

Weights before & after L^2

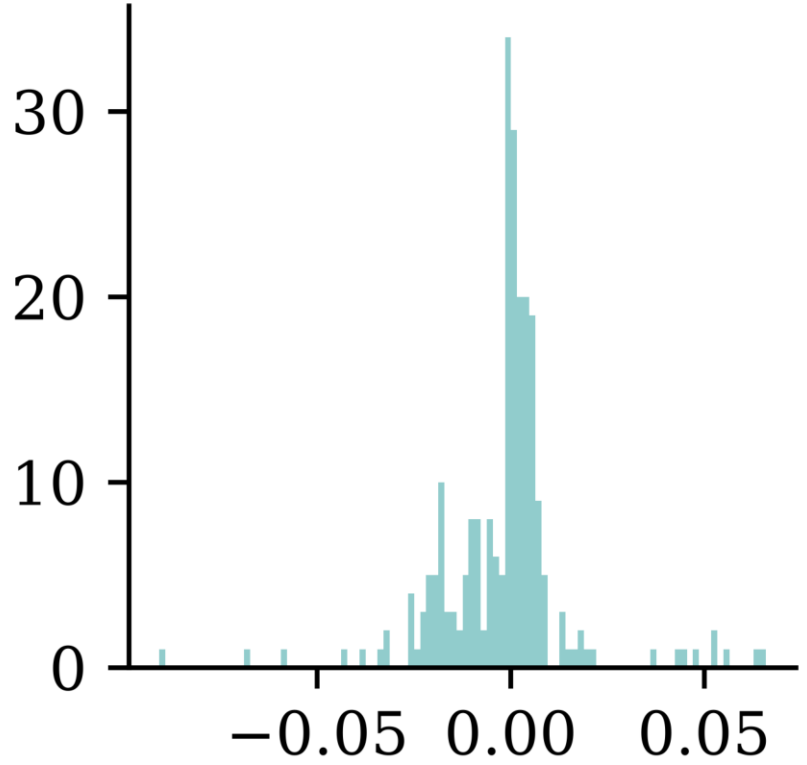
```
model = l2_model(0.0)
weights = model.layers[0].get_weights()[0
print(f"Number of weights almost 0: {np.s
plt.hist(weights, bins=100);
```

Number of weights almost 0: 0




```
model = l2_model(1.0)
weights = model.layers[0].get_weights()[0
print(f"Number of weights almost 0: {np.s
plt.hist(weights, bins=100);
```

Number of weights almost 0: 6

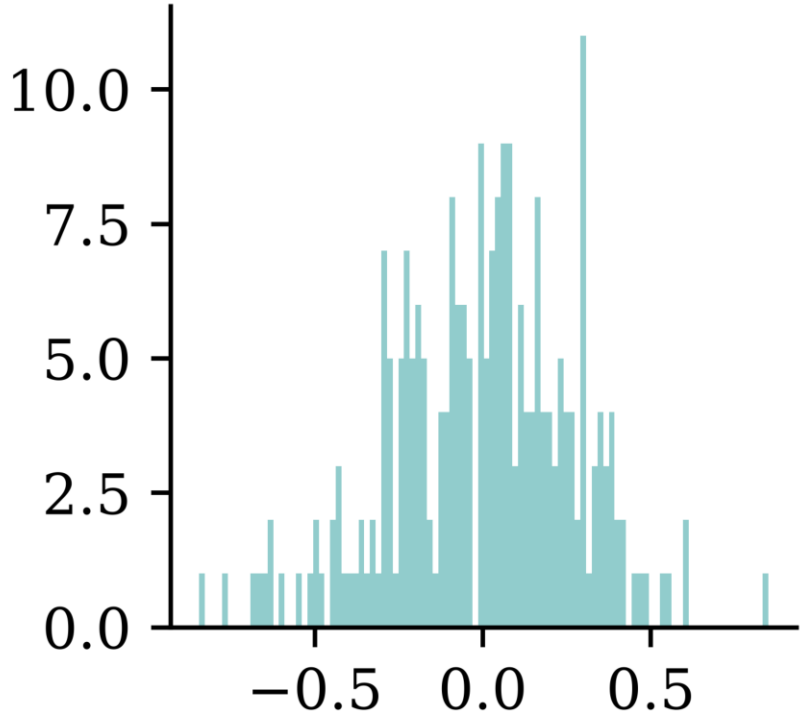


UNSW SYDNEY

Weights before & after L^1

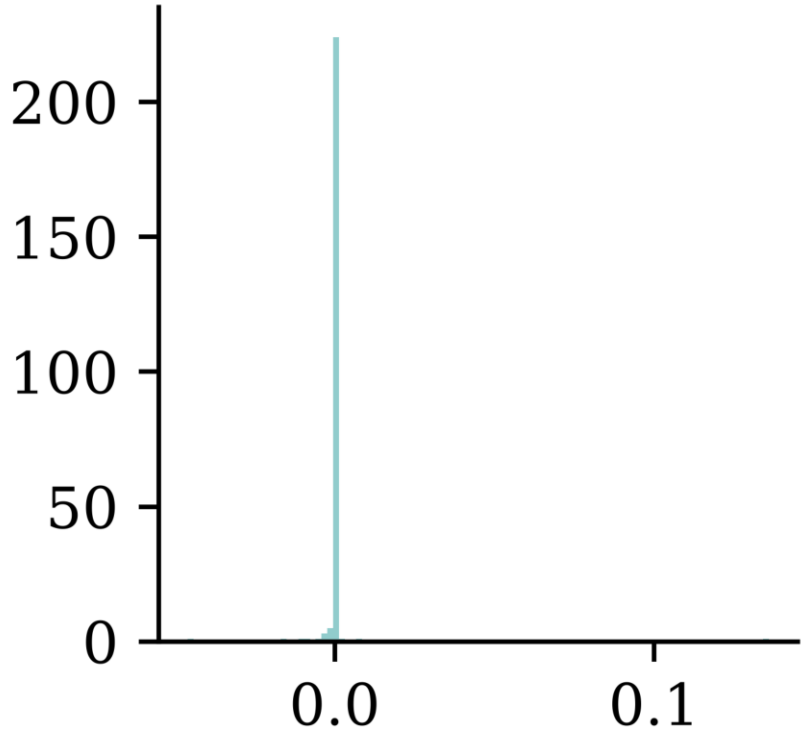
```
1 model = l1_model(0.0)
2 weights = model.layers[0].get_weights()[0
3 print(f"Number of weights almost 0: {np.s
4 plt.hist(weights, bins=100);
```

Number of weights almost 0: 0




```
1 model = l1_model(1.0)
2 weights = model.layers[0].get_weights()[0
3 print(f"Number of weights almost 0: {np.s
4 plt.hist(weights, bins=100);
```

Number of weights almost 0: 12



Early-stopping regularisation

> A very different way to regularize iterative learning algorithms such as gradient descent is to stop training as soon as the validation error reaches a minimum. This is called early stopping… It is such a simple and efficient regularization technique that Geoffrey Hinton called it a "beautiful free lunch".

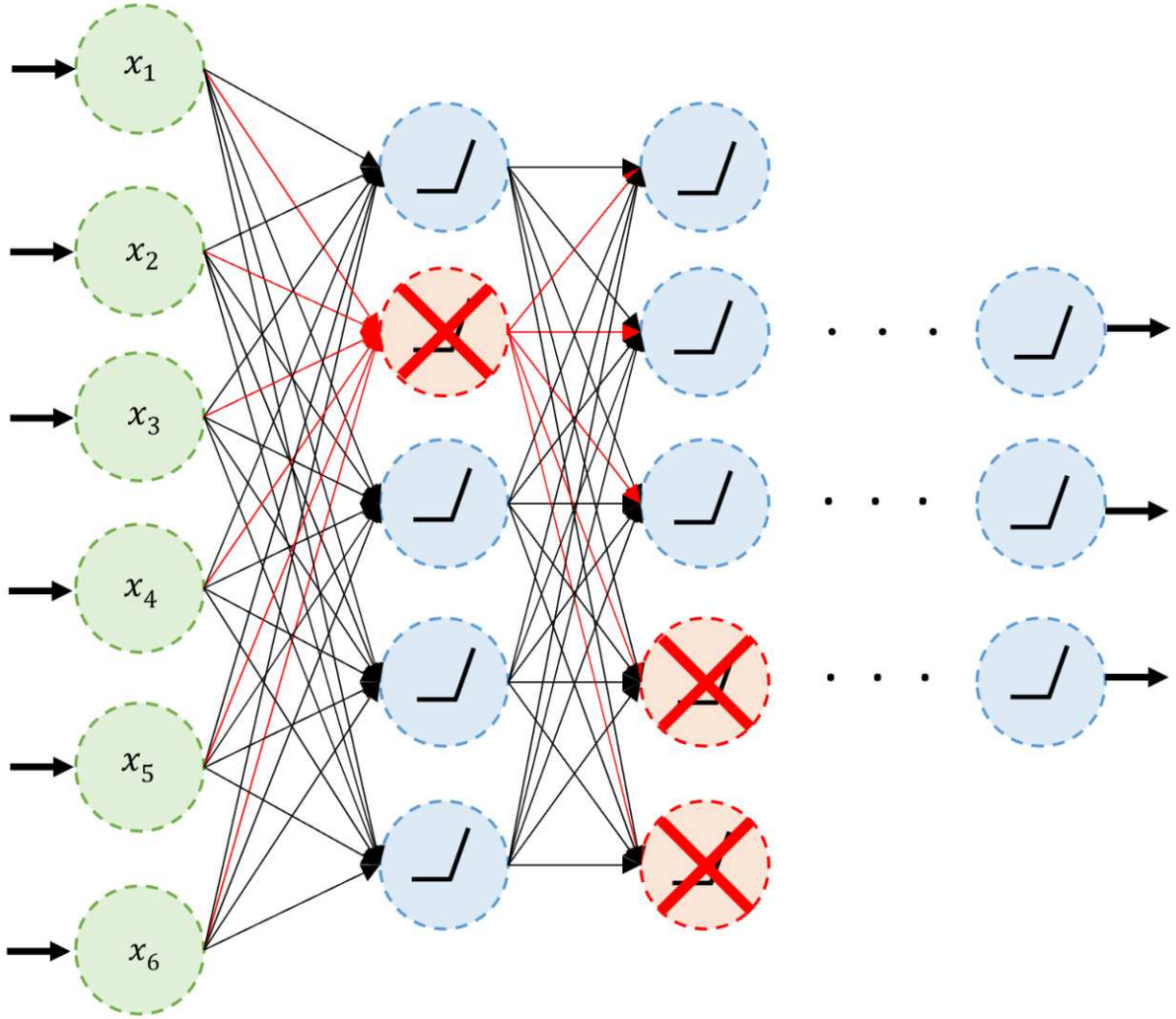
> Alternatively, you can try building a model with slightly more layers and neurons than you actually need, then use early stopping and other regularization techniques to prevent it from overfitting too much. Vincent Vanhoucke, a scientist at Google, has dubbed this the "stretch pants" approach: instead of wasting time looking for pants that perfectly match your size, just use large stretch pants that will shrink down to the right size.

Source: Géron (2019), Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Edition, Chapters 4 and 10.

Lecture Outline

- Introduction
- Traditional Regression
- Stochastic Forecasts
- GLMs and Neural Networks
- Combined Actuarial Neural Network
- Mean-Variance Estimation Network
- Mixture Density Network
- Distributional Refinement Network
- Uncertainty and Regularisation
- **Dropout and Ensembles**

Dropout



An example of neurons dropped during training.

Sources: Marcus Lautier (2022).

Dropout quote

> It's surprising at first that this destructive technique works at all. Would a company perform better if its employees were told to toss a coin every morning to decide whether or not to go to work? Well, who knows; perhaps it would! The company would be forced to adapt its organization; it could not rely on any single person to work the coffee machine or perform any other critical tasks, so this expertise would have to be spread across several people. Employees would have to learn to cooperate with many of their coworkers, not just a handful of them. The company would become much more resilient. If one person quit, it wouldn't make much of a difference. It's unclear whether this idea would actually work for companies, but it certainly does for neural networks. Neurons trained with dropout cannot co-adapt with their neighboring neurons; they have to be as useful as possible on their own. They also cannot rely excessively on just a few input neurons; they must pay attention to each of their input neurons. They end up being less sensitive to slight changes in the inputs. In the end, you get a more robust network that generalizes better.

Source: Aurélien Géron (2019), *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd Edition, p. 366

Dropout

Dropout is just another layer in Keras.

```
random.seed(2);

model = Sequential([
    Dense(30, activation="leaky_relu"),
    Dropout(0.2),
    Dense(30, activation="leaky_relu"),
    Dropout(0.2),
    Dense(1, activation="exponential")
])

model.compile("adam", "mse")
model.fit(X_train_sc, y_train, epochs=4, verbose=0);
```

Dropout after training

Making predictions is the same as any other model:

```
model.predict(X_train_sc.head(3),
                    verbose=0)
```

```
array([[1.73],
       [0.74],
       [1.56]], dtype=float32)
```

```
model.predict(X_train_sc.head(3),
                    verbose=0)
```

```
array([[1.73],
       [0.74],
       [1.56]], dtype=float32)
```

We can make the model think it is still training:

```
keras.ops.convert_to_numpy(
    model(X_train_sc.head(3), training=Tr
```

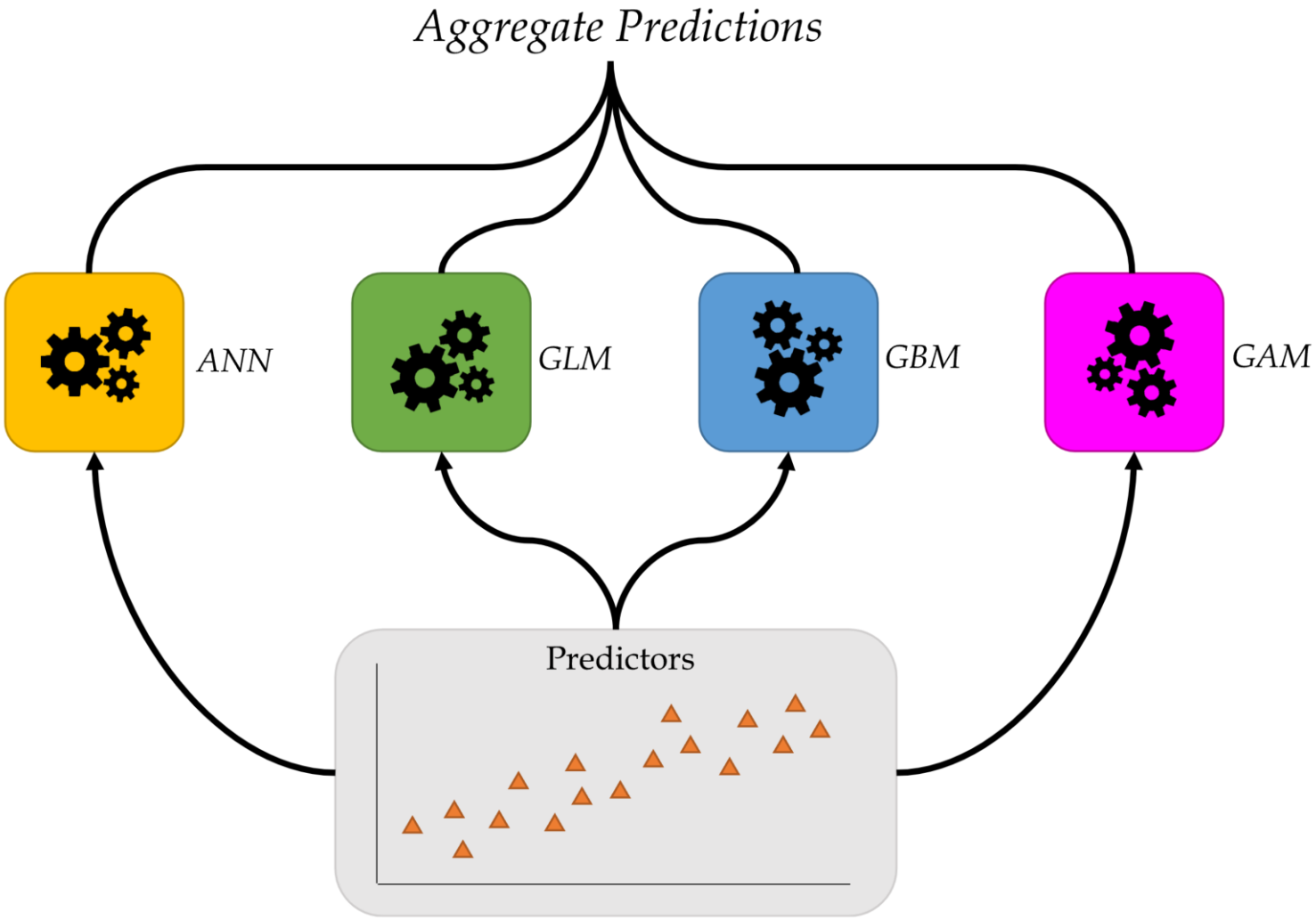
```
array([[2.08],
       [0.85],
       [1.43]], dtype=float32)
```

```
keras.ops.convert_to_numpy(
    model(X_train_sc.head(3), training=Tr
```

```
array([[1.71],
       [0.6 ],
       [1.51]], dtype=float32)
```

By setting the `training=True`, we can let dropout happen during prediction stage as well. This will change predictions for the same output differently. This is known as the *Monte Carlo dropout* and can be used to generate a distribution of predictions.

Ensembles



Combine many models to get better predictions.

Source: Marcus Lautier (2022).

Deep Ensembles

Train M neural networks with different random initial weights independently (even in parallel).

```python
def build_model(seed):
    random.seed(seed)
    model = Sequential([
        Dense(30, activation="leaky_relu"),
        Dense(1, activation="exponential")
    ])
    model.compile("adam", "mse")

    es = EarlyStopping(restore_best_weights=True, patience=5)
    model.fit(X_train_sc, y_train, epochs=1_000,
        callbacks=[es], validation_data=(X_val_sc, y_val), verbose=False)
    return model
```

```python
M = 3
seeds = range(M)
models = []
for seed in seeds:
    models.append(build_model(seed))
```

Deep Ensembles II

Say the trained weights for the NNs are $\boldsymbol{w}^{(1)}, \ldots, \boldsymbol{w}^{(M)}$, then we get predictions $\left\{\hat{y}(\boldsymbol{x}; \boldsymbol{w}^{(m)})\right\}_{m=1}^{M}$

```
y_preds = []
for model in models:
    y_preds.append(model.predict(X_test_sc, verbose=0))

y_preds = np.array(y_preds)
y_preds
```

```
array([[[3.36],
        [0.79],
        [2.33],
        ...,
        [2.42],
        [2.43],
        [1.09]],

       [[1.66],
        [1.13],
        [1.72],
        ...,
        [1.91],
        [1.94],
        [2.34]],
```

Package Versions

```
1 from watermark import watermark
2 print(watermark(python=True, packages="keras,matplotlib,numpy,pandas,seaborn,scipy,torch,
```

```
Python implementation: CPython
Python version       : 3.14.5
IPython version      : 9.15.0

keras     : 3.15.0
matplotlib: 3.11.0
numpy     : 2.5.0
pandas    : 3.0.3
seaborn   : 0.13.2
scipy     : 1.18.0
torch     : 2.12.1
drn       : 1.0.0
```

Glossary

- aleatoric and epistemic uncertainty
- Combined Actuarial Neural Network
- Deep GLM
- Deep Ensembles
- distributional forecasts
- Distributional Refinement Network
- dropout
- Mixture Density Network
- mixture distribution
- Monte Carlo dropout

References

Al-Mudafer, M. T., Avanzi, B., Taylor, G., & Wong, B. (2022). Stochastic loss reserving with mixture density neural networks. *Insurance: Mathematics and Economics*, *105*, 144–174.

Avanzi, B., Dong, E. T., Laub, P. J., & Wong, B. (2026). Distributional refinement network: Distributional forecasting via deep learning. *Insurance: Mathematics and Economics*, *128*, 103246.

Bishop, C. M. (1994). *Mixture density networks*.

Delong, Ł., Lindholm, M., & Wüthrich, M. V. (2021). Gamma mixture density networks and their application to modelling insurance claim amounts. *Insurance: Mathematics and Economics*, *101*, 240–261.

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *International Conference on Machine Learning*, 1321–1330.

Nix, D. A., & Weigend, A. S. (1994). Estimating the mean and variance of the target probability distribution. *Proceedings of 1994 Ieee International Conference on Neural Networks (ICNN'94)*, *1*, 55–60.

Schelldorfer, J., & Wüthrich, M. V. (2019). Nesting classical actuarial models into neural networks. *Available at SSRN 3320525*.

Tran, M.-N., Nguyen, N., Nott, D., & Kohn, R. (2020). Bayesian deep net GLM and GLMM. *Journal of Computational and Graphical Statistics*, *29*(1), 97–113.